SISTEMAS OPERATIVOS

UNIDAD VI

Contenidos

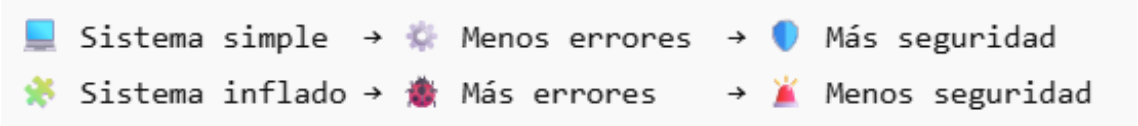
Aspectos del entorno de seguridad. 3
Objetivos y amenazas. 4
Intrusos. 6
Pérdida accidental de datos. 8
Referencias y bibliografía de la sección. 9
Fundamentos de la criptografía (cifrado). 9
Criptografía de clave secreta. 10
Criptografía de clave pública. 12
Funciones de una vía y hashing. 14
Firmas digitales. 15
Módulo de plataforma confiable (TPM). 18
Referencias y bibliografía de la sección. 20
Mecanismos de protección. 20
Dominios de protección. 20
Listas de control de acceso. 22
Capacidades. 26
Sistemas confiables. 27
Base de cómputo confiable. 27
Referencias y bibliografía de la sección. 29
Autenticación, identificación y validación de usuarios. 29
Autenticación mediante el uso de contraseñas. 31
Seguridad de contraseñas de UNIX. 32
Contraseñas de un solo uso. 34
Autenticación mediante el uso de un objeto físico. 35
Autenticación mediante biométrica. 36
Referencias y bibliografía de la sección. 38
Defensas. 38
Firewalls. 39
Los antivirus y las técnicas anti-antivirus. 41
Exploradores de virus. 42

Comprobadores de integridad. 44
Comprobadores del comportamiento. 45
Cómo evitar los virus. 46
Firma de código. 46
Encarcelamiento 48
Detección de intrusos basada en modelos. 48
Referencias y bibliografía de la sección. 50

Aspectos del entorno de seguridad.

Muchas empresas poseen información valiosa que desean tener muy bien protegida. Lo mismo pasa con las computadoras domésticas que contienen datos valiosos. Por lo tanto, proteger esta información contra el uso no autorizado es una de las principales preocupaciones de todos los sistemas operativos.

Por desgracia, también es cada vez más difícil hacer esto debido a la amplia aceptación de los sistemas inflados (y los errores que los acompañan) como un fenómeno normal. Software con una gran cantidad de funciones innecesarias, dependencias y código redundante incrementa el riesgo de errores y vulnerabilidades de seguridad.



Las cuestiones relacionadas con la seguridad de los sistemas operativos han cambiado de manera radical en las últimas dos décadas. No fue sino hasta principios de la década de 1990 que algunas personas tenían una computadora en su casa, y la mayoría de los trabajos de cómputo se realizaban en empresas, universidades y otras organizaciones en computadoras multiusuario, desde grandes mainframes hasta minicomputadoras. Casi todas estas máquinas estaban aisladas, sin conexión a una red.

Como consecuencia, la seguridad estaba enfocada casi por completo en la forma de evitar que los usuarios se entrometieran en los asuntos de los demás usuarios.

De esta forma, si un sistema operativo tenía dos usuarios registrados, el truco era asegurar que ninguna de los dos pudiera leer los archivos del otro, o modificarlos, pero permitir que compartieran los archivos que querían compartir.

Se desarrollaron modelos y mecanismos elaborados para asegurar que ningún usuario pudiera obtener derechos de acceso a los archivos que no debía. Algunas veces, los modelos y los mecanismos implicaban clases de usuarios en vez de sólo individuos. Por ejemplo, en una computadora militar los datos tenían que marcarse como muy secretos, secretos, confidenciales o públicos, y había que evitar que los cabos husmearan en los directorios de los generales, sin importar quién era el cabo y quién era el general.

Todos estos temas se investigaron de manera detallada, se hicieron reportes sobre ellos y se implementaron durante varias décadas. Una suposición tácita era que, una vez que se elegía el modelo y se realizaba la implementación, el software era básicamente correcto y cumpliría con todas las reglas. Por lo general, los modelos y el software eran bastante simples, por lo que la suposición casi siempre era válida.

La situación cambió con el surgimiento de la computadora personal e Internet, y con la desaparición de la mainframe y las minicomputadoras compartidas. Por desgracia, surgió otra amenaza: los ataques del exterior. Surgieron virus, gusanos y otras plagas digitales, entraron a

las computadoras por medio de Internet y una vez establecidos, causaron todo tipo de estragos.

Lo que los ayudó a hacer daño fue el explosivo crecimiento del bugware inflado, que ha sustituido al formidable software eficiente de los años anteriores. Ahora que los sistemas operativos contienen más de 5 millones de líneas de código en el kernel y las aplicaciones grandes son la regla en vez de la excepción, hay grandes cantidades de errores que las plagas digitales pueden explotar para hacer cosas que las reglas no permiten.

- **Ejemplo histórico:**
- **Melissa (1999)**: Gusano que se propagaba por correo electrónico, explotando vulnerabilidades en Microsoft Word y Outlook.
- **Code Red (2001)**: Gusano que atacaba servidores Windows, explotando errores en IIS.

Por lo tanto, ahora tenemos una situación en la que podemos mostrar formalmente que un sistema es seguro, y aun así se puede ver comprometido con facilidad debido a que cierto error en el código permite que un programa salvaje haga cosas que formalmente tiene prohibido hacer.

Para evitar una confusión, utilizaremos el término seguridad para hacer referencia al problema general, y mecanismos de protección para referirnos a los mecanismos específicos del sistema operativo que se utilizan para salvaguardar la información en la computadora. Sin embargo, no hay un límite definido entre estos dos términos.

La seguridad tiene muchas facetas. Tres de las más importantes son la naturaleza de las amenazas, la naturaleza de los intrusos y la pérdida accidental de datos.

Objetivos y amenazas.

Desde la perspectiva de la ***seguridad***, en sistemas computacionales se tienen cuatro objetivos generales con sus correspondientes amenazas, como se muestra en la lista de la figura que tenemos a continuación.

Objetivo	Amenaza
Confidencialidad de los datos	Exposición de los datos
Integridad de los datos	Alteración de los datos
Disponibilidad del sistema	Negación del servicio
Exclusión de los usuarios externos	Los virus se apropian del sistema

Ilustración 1 - Objetivos y amenazas de seguridad.

El primer objetivo, conocido como ***confidencialidad de los datos***, implica hacer que los datos secretos permanezcan así. Por ejemplo, si el propietario de ciertos datos ha decidido que éstos pueden estar disponibles sólo para ciertas personas, el sistema debe garantizar que las personas no autorizadas nunca tengan acceso a esos datos. Como un mínimo absoluto, el propietario debe ser capaz de especificar quién puede ver qué cosa, y el sistema debe cumplir con estas especificaciones, que en teoría se deben aplicar a cada archivo en forma individual.

El segundo objetivo, conocido como ***integridad de los datos***, significa que los usuarios sin autorización no deben ser capaces de modificar datos sin el permiso del propietario. La modificación de datos en este contexto incluye no sólo la modificación de los datos, sino también su eliminación y la inclusión de datos falsos. Si un sistema no puede garantizar que los datos depositados en él permanecerán sin modificación hasta que el usuario decida hacerlo, no tiene mucho valor como sistema de información.

El tercer objetivo, conocido como ***disponibilidad del sistema***, significa que nadie puede perturbar el sistema para hacerlo inutilizable. Dichos ataques de negación del servicio son muy comunes. Por ejemplo, si una computadora es un servidor de Internet y alguien le envía una avalancha de peticiones, puede dejarlo inhabilitado al ocupar todo el tiempo de su CPU con tan sólo tener que examinar y descartar las peticiones entrantes. Si, por ejemplo, requiere 100 µseg para procesar una petición entrante para leer una página Web, entonces cualquiera que se las arregle para enviar 10,000 peticiones/segundo podrá aniquilarlo.

Hay disponibles modelos razonables y tecnología para lidiar con los ataques sobre la confidencialidad y la integridad; es mucho más difícil frustrar estos ataques de negación de servicio.

Tipo de Perturbación	Recurso Afectado	Solución Integrada del Kernel
Bucle infinito / Bloqueo	CPU	Planificación apropiativa (Interrupciones de reloj).
Saturación de almacenamiento	Disco Secundario	Límites y cuotas en el Sistema de Archivos.
Agotamiento de espacio	Memoria Principal	Tablas de páginas, límites de proceso y swap.

El ultimo objetivo es la ***exclusión de los usuarios externos*** que pueden tomar el control de las computadoras en el hogar de otras personas (mediante el uso de virus y otros medios) y convertirlas en zombies, dispuestas a cumplir los deseos del usuario exterior con sólo dar las órdenes.

A menudo los zombies se utilizan para enviar spam, de manera que no se pueda rastrear a la persona que está detrás del ataque de spam.

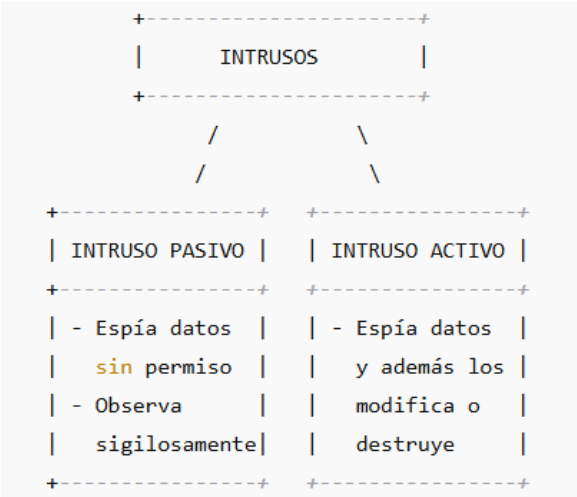
Amenaza Técnica	Mecanismo del Atacante	¿Cómo se defiende el Sistema Operativo?
Zombificación (Botnet)	Ejecución de procesos ocultos que responden a un servidor externo.	Monitoreo de conexiones de red, firmas de código y aislamiento de procesos en memoria principal.
Violación de Privacidad	Acceso no autorizado a archivos personales o intercepción de datos.	Cifrado nativo de archivos (como BitLocker o eCryptfs) y Listas de Control de Acceso (ACL).

Otro aspecto del problema de seguridad es la ***privacidad***: proteger a los individuos contra el mal uso de la información sobre ellos. Esto ha generado evidentes problemas legales y morales.

Intrusos.

En la literatura de la seguridad, las personas que husmean en lugares en donde no tienen por qué hacerlo se conocen como ***intrusos***, o algunas veces como adversarios.

Los intrusos actúan en dos formas distintas. Los ***intrusos pasivos*** sólo quieren leer archivos para los cuales no tienen autorización. Los ***intrusos activos*** son más maliciosos; desean realizar modificaciones no autorizadas a los datos.



Al diseñar un sistema para que sea seguro contra los intrusos, es importante tener en cuenta el tipo de intruso contra el que tratamos de protegerlo. Algunas categorías comunes son:

1. **Usuarios no técnicos que se entrometen en forma casual.** Muchas personas tienen computadoras personales, las cuales están conectadas a un servidor de archivos compartidos

y, debido a la naturaleza curiosa de los humanos, algunas de esas personas son capaces de leer el correo electrónico y demás archivos de otras si no hay barreras que las detengan. Por ejemplo, la mayoría de los usuarios de UNIX tienen la opción predeterminada de que todos los archivos recién creados tienen permisos de lectura para todos.

2. **Intrusos que husmean.** Los estudiantes, programadores de sistemas, operadores y demás personal técnico a menudo consideran como un reto personal la acción de irrumpir en la seguridad de un sistema computacional local. Por lo general son muy habilidosos y están dispuestos a dedicar una cantidad considerable de tiempo a ello.

- **Perfil:** Estudiantes de informática, programadores de sistemas o personal operativo con acceso legítimo básico al entorno de hardware.
- **Motivación:** Curiosidad, ego, reto personal o la búsqueda de fallas de diseño dentro del propio software del sistema operativo.
- **Ventaja:** Al estar dentro de la organización, conocen las debilidades físicas y lógicas locales, lo que les permite intentar escaladas de privilegios desde el modo usuario hacia el modo kernel.

3. **Intentos determinados por obtener dinero.** Algunos programadores de los bancos han tratado de robar del banco en el que trabajan. Los esquemas varían, desde cambiar el software para truncar en vez de redondear el interés, quedarse con la fracción de un centavo, desviar las cuentas que no se han utilizado en años, hasta llegar al chantaje (“Si no me pagan, destruiré todos los registros del banco”).

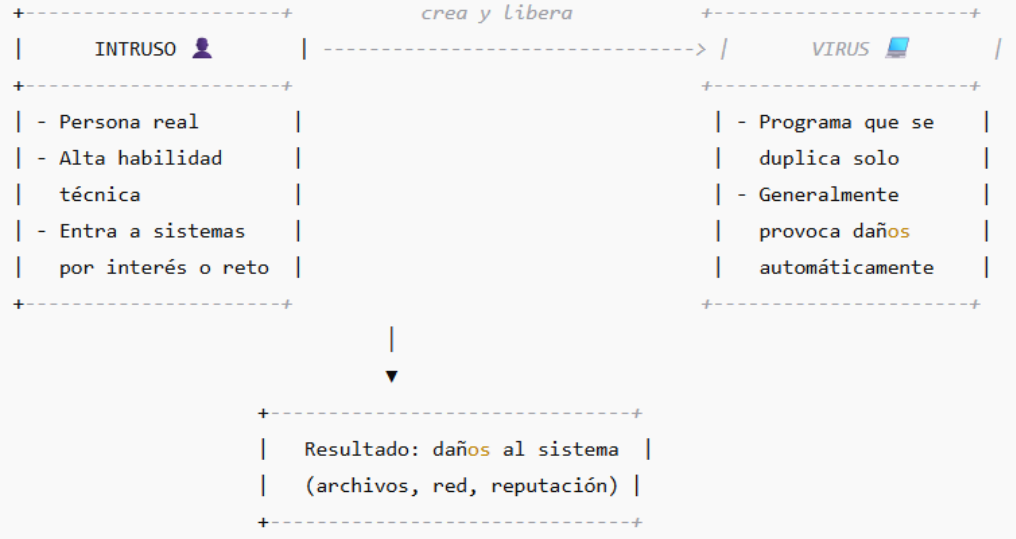
4. **Espionaje comercial o militar.** El espionaje se refiere a un intento serio y bien fundamentado por parte de un competidor u otro país de robar programas, secretos comerciales, ideas patentables, tecnología, diseños de circuitos, planes de negocios, etcétera. A menudo para hacer esto se intervienen líneas telefónicas o incluso se montan antenas dirigidas hacia la computadora para recoger su radiación electromagnética.

Es evidente que tratar de evitar que un gobierno extranjero hostil robe secretos militares es algo muy distinto a tratar de evitar que los estudiantes inserten un mensaje gracioso en el sistema. La cantidad de esfuerzo necesaria para la seguridad y la protección depende sin duda de quién creamos que es el enemigo.

El ***virus*** es otra categoría de plaga de seguridad. En esencia, un virus es una pieza de código que se duplica a sí mismo y (por lo general) realiza cierto daño. En cierto modo, el escritor de un virus es también un intruso, a menudo con habilidades técnicas elevadas.

¿Qué es?	Código malicioso que se copia a sí mismo para causar daños.
¿Cómo actúa?	Puede ocultarse dentro de un programa sano, infectar sectores de arranque del disco o activarse solo cuando se cumple una condición (como una fecha).
Sus objetivos	Varían según el creador: desde mostrar mensajes molestos y consumir recursos, hasta borrar archivos, cifrar el disco o robar información.

La diferencia entre un intruso convencional y un virus es que el primero se refiere a una persona que trata de irrumpir en un sistema por motivos personales para ocasionar daños, mientras que el segundo es un programa escrito por dicha persona y que después se suelta en el mundo con la esperanza de que provoque daños. Los intrusos tratan de irrumpir en sistemas específicos (por ejemplo, uno que pertenezca a cierto banco o al Pentágono) para robar o destruir datos específicos, mientras que el escritor de un virus comúnmente quiere provocar daños en general, y no le importa a quién.



Pérdida accidental de datos.

Además de las amenazas ocasionadas por los intrusos maliciosos, por accidente se pueden perder datos valiosos. Algunas de las causas comunes de pérdida accidental de datos son:

1. **Accidentes y desastres naturales:** incendios, inundaciones, terremotos, guerras, disturbios o ratas que roen cintas magnéticas.
2. **Errores de hardware o software:** fallas en la CPU, discos o cintas que no se pueden leer, errores de telecomunicaciones, errores en los programas.

3. **Errores humanos:** error al introducir los datos, al montar una cinta o un CD-ROM de manera incorrecta; ejecutar el programa incorrecto, perder un disco o una cinta, o cualquier otro error.

La mayoría de estas causas se pueden prevenir mediante la realización de ***respaldos*** adecuados que se guardan, de preferencia, lejos de los datos originales. Aunque la acción de proteger los datos contra pérdidas accidentales puede parecer mundana en comparación con la acción de proteger contra intrusos astutos, en la práctica es probable que haya más daños ocasionados por las pérdidas accidentales que por intrusos.

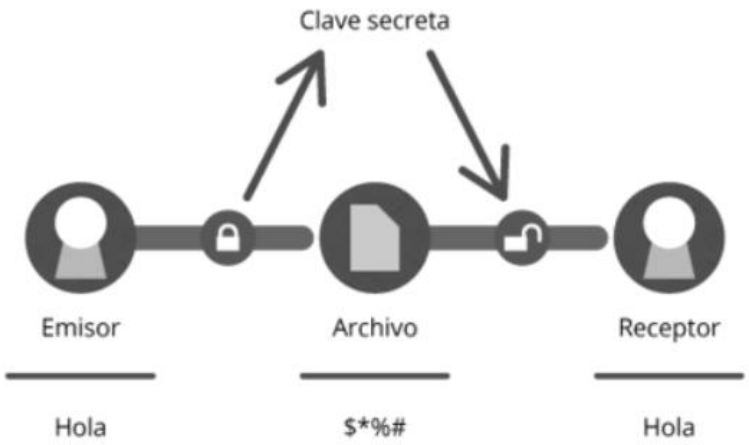
Referencias y bibliografía de la sección.

- Tanenbaum 9-1.

Fundamentos de la criptografía (cifrado).

La criptografía desempeña un importante papel en la seguridad. Muchas personas están familiarizadas con los criptogramas de los periódicos, que son pequeños rompecabezas en donde cada letra se ha sustituido de manera sistemática por una letra distinta. En realidad, no tienen mucho que ver con la criptografía moderna.

El propósito de la criptografía es tomar un mensaje o archivo, conocido como texto simple, y convertirlo en texto ***cifrado*** de tal forma que sólo las personas autorizadas sepan cómo convertirlo nuevamente en texto simple. Para todos los demás, el texto cifrado es sólo una sucesión incomprensible de bits.



Aunque pueda parecer extraño para los principiantes en esta área, los algoritmos de cifrado y descifrado (funciones) siempre deben ser públicos.

El secreto depende de los parámetros de los algoritmos, a los cuales se les denomina ***claves***. Si *P* es el archivo de texto simple, *KE* es la clave de cifrado, *C* es el texto cifrado y *E* es el algoritmo de cifrado (es decir, la función), entonces *C E (P, KE)*. Ésta es la definición del cifrado, la cual indica que el texto cifrado se obtiene mediante el uso del algoritmo de cifrado *E*, con el texto simple *P* y la clave de cifrado (secreta) *KE* como parámetros.

La idea de que todos los algoritmos deben ser públicos y el secreto debe residir exclusivamente en las claves se conoce como ***Principio de Kerckhoffs***, formulado por el criptógrafo Auguste Kerckhoffs[1] del siglo XIX.

De manera similar tenemos que *P D (C, KD),* donde *D* es el algoritmo de descifrado y *KD* es la clave de descifrado. Esto indica que para obtener de vuelta el texto simple *P* a partir del texto cifrado *C* y la clave de descifrado *KD*, hay que ejecutar el algoritmo *D* con *C* y *KD* como parámetros. En la figura siguiente se muestra la relación entre las diversas piezas.

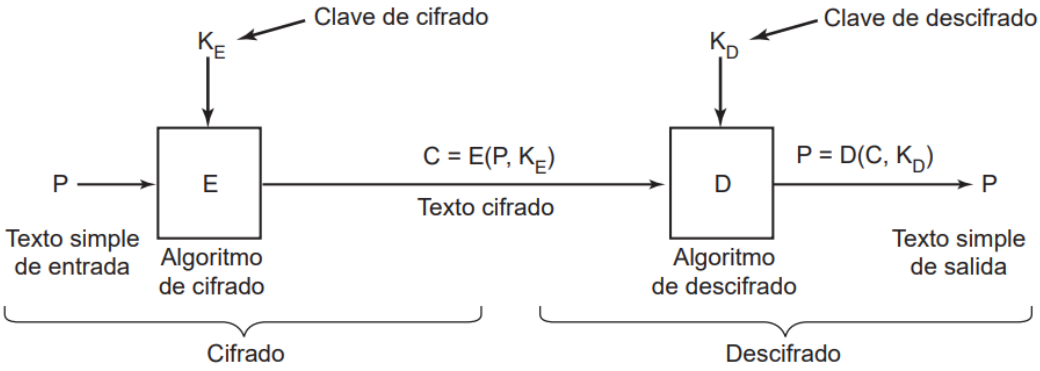


Ilustración 2 - Relación entre el texto simple y el texto cifrado.

Criptografía de clave secreta.

La ***criptografía de clave secreta*** (también llamada simétrica) usa la misma clave para cifrar y descifrar datos.

Para que esto sea más claro, consideremos un algoritmo en el que cada letra se sustituye por una letra distinta; por ejemplo, todas las A se sustituyen por Q, todas las B se sustituyen por W, todas las C se sustituyen por E, y así en lo sucesivo:

texto simple: A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
texto cifrado: Q W E R T Y U I O P A S D F G H J K L Z X C V B N M

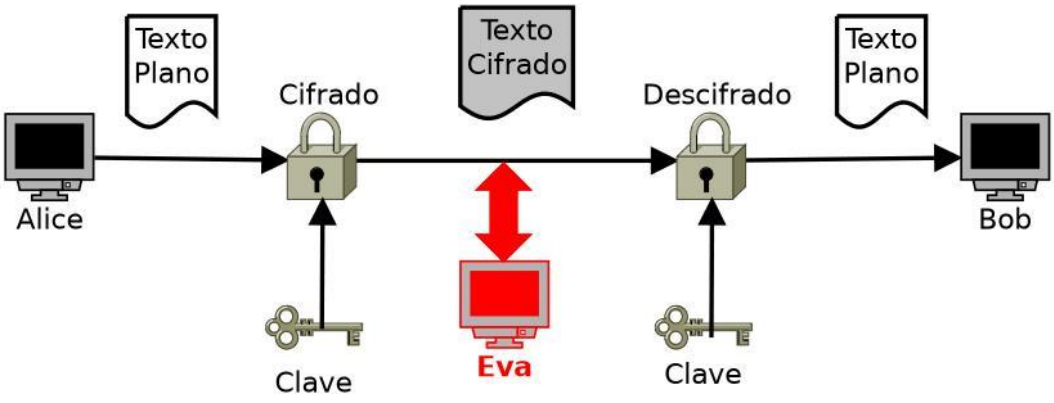
[1] El principio de Kerckhoffs (también llamado desiderátum, supuesto, axioma, doctrina o ley de Kerckhoffs) de la criptografía fue enunciado por el criptógrafo holandés Auguste Kerckhoffs en el siglo XIX. Este principio sostiene que un criptosistema debe ser seguro, incluso si todo lo relacionado con el sistema, excepto la clave, es de dominio público. Este concepto es ampliamente aceptado por los criptógrafos, a diferencia de la seguridad por oscuridad, que no lo es.

A este sistema general se le conoce como sustitución ***monoalfabética***, en donde la clave es la cadena de 26 letras correspondiente al alfabeto completo.

La clave de cifrado en este ejemplo es *QWERTYUIOPASDFGHJKLZXCVBNM*. Utilizando la clave anterior, el texto simple *ATTACK* se transformaría en el texto cifrado *QZZQEA*.

La clave de descifrado indica cómo obtener de vuelta el texto simple a partir del texto cifrado. En este ejemplo, la clave de descifrado es *KXVMCNOPHQRSZYIJADLEGWBUFT*, debido a que una A en el texto cifrado es una K en el texto simple, una B en el texto cifrado es una X en el texto simple, y así sucesivamente.

En primera instancia esto podría parecer un sistema seguro, pues, aunque el criptoanalista conozca el sistema general (sustitución de letra por letra), no sabe cuál de las posibles claves se está utilizando. Sin embargo, cuando el texto cifrado es muy poco, la clave se puede descubrir con facilidad.



El ataque básico saca provecho de las propiedades estadísticas de los lenguajes naturales. Por ejemplo, en inglés la letra *e* es la más común, seguida de *t*, *o*, *a*, *n*, *i*, etc. Las combinaciones más comunes de dos letras, conocidas como bigramas, son *th*, *in*, *er*, *re* y así, en lo sucesivo.

Es fácil quebrantar el cifrado si se utiliza este tipo de información. Muchos sistemas criptográficos como el anterior tienen la propiedad de que, dada la clave de cifrado, es fácil encontrar la clave de descifrado, y viceversa. A dichos sistemas se les conoce como ***criptografía de clave secreta*** o ***criptografía de clave simétrica***.

Aunque los cifrados de sustitución monoalfabética son completamente inútiles, hay otros algoritmos de clave simétrica conocidos que son relativamente seguros si las claves son lo bastante largas. Las claves más cortas pueden frustrar a los amateurs, pero no a los principales gobiernos.

Mecanismo	Utiliza la misma clave tanto para cifrar como para descifrar.
Ventaja Principal	Extremadamente rápida y eficiente para procesar grandes volúmenes de datos.
Desventaja Crítica	Complejidad absoluta para compartir y proteger la clave de forma segura.
Riesgo en textos cortos	Mayor facilidad para que un criptoanalista descubra la clave.

Criptografía de clave pública.

Los sistemas de clave secreta son eficientes debido a que el monto de cálculos requeridos para cifrar o descifrar un mensaje es razonable, pero hay una gran desventaja: el emisor y el receptor deben tener la clave secreta compartida.

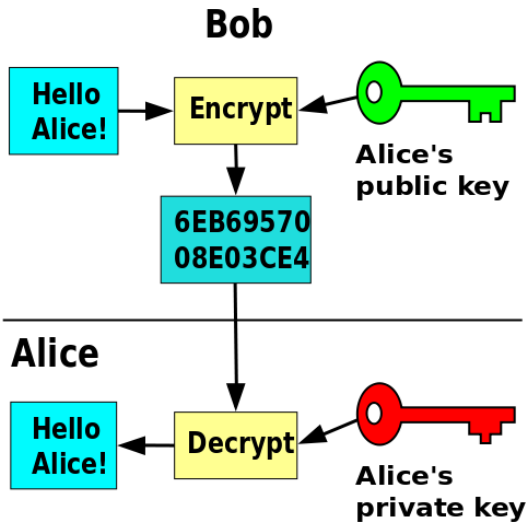
Para resolver este problema se utiliza la criptografía de clave pública. Este sistema tiene la propiedad de que se utilizan distintas claves para el cifrado y el descifrado, y si se elige bien la clave de cifrado es casi imposible descubrir la clave de descifrado correspondiente.

Bajo estas circunstancias, la clave de cifrado se puede hacer pública y sólo hay que mantener secreta la clave de descifrado privada. Para ver cómo funciona la criptografía de clave pública, consideremos las siguientes dos preguntas:

Pregunta 1: ¿Cuánto es $314159265358979 \times 314159265358979$?
Pregunta 2: ¿Cuál es la raíz cuadrada de $3912571506419387090594828508241$?

La pregunta 1 es relativamente fácil de responder. Se necesitaría una calculadora, computadora u otro tipo de ayuda externa para resolver la pregunta 2.

Aunque el cuadrado y la raíz cuadrada son operaciones inversas, tienen grandes diferencias en cuanto a su complejidad computacional. Este tipo de asimetría forma la base de la criptografía de clave pública.



El cifrado utiliza una operación sencilla, pero para el descifrado sin la clave se necesita la operación difícil. Hay un sistema de clave pública llamado ***RSA***, el cual explota el hecho de que para una computadora es mucho más fácil multiplicar números muy grandes que obtener la factorial de números muy grandes, en especial cuando todas las operaciones aritméticas se realizan mediante la aritmética de módulo y todos los números involucrados tienen cientos de dígitos.

El principal problema con la criptografía de clave pública es que es mucho más lenta que la criptografía simétrica.

Criterio	Características
¿Qué es?	Un sistema que usa dos llaves distintas para proteger la información: una pública y una privada.
¿Cómo funciona?	Lo que se cifra con la llave pública solo se puede descifrar con su llave privada correspondiente.
Su seguridad	Descubrir la llave privada desde la pública es tan difícil matemáticamente que a una computadora le tomaría miles de años de trabajo continuo lograrlo.
Su gran ventaja	Resuelve el problema del sistema simétrico, porque ya no hace falta que las dos personas compartan una misma clave secreta de antemano.

El funcionamiento de la criptografía de clave pública es el siguiente: todos eligen un par (clave pública, clave privada) y publican la clave pública. Esta clave pública es la clave de cifrado; la clave privada es la clave de descifrado. Por lo general, el proceso de generación de la clave es

automatizado, en donde tal vez una contraseña seleccionada por el usuario alimenta al algoritmo como una semilla.

Para enviar un mensaje secreto a un usuario, una corresponsal cifra el mensaje con la clave pública del receptor. Como sólo el receptor tiene la clave privada, sólo él puede descifrar el mensaje.



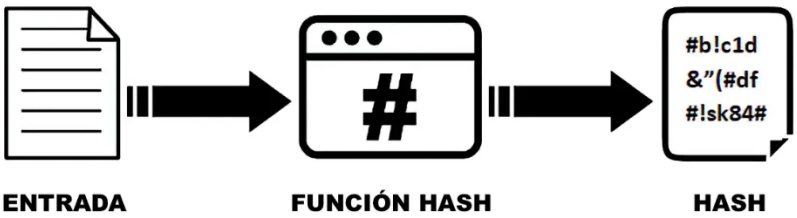
Funciones de una vía y hashing.

Desde un punto de vista amplio, una función de una vía es cualquier función que es fácil de calcular en un sentido, pero extremadamente difícil (inviable) de revertir.

Hay varias situaciones donde es deseable tener cierta función *f* con la siguiente propiedad: dada *f* y su parámetro *x*, es fácil calcular *y = f(x)*, pero si sólo se tiene *f(x)* es imposible calcular el valor de *x*.

En criptografía, comúnmente, dicha función de una vía manipula los bits en formas complejas. Podría empezar por inicializar *y* con *x*. Después podría ejecutar un ciclo que itere todas las veces que haya bits 1 en *x*, y en cada iteración se permutarían los bits de *y* de una manera independiente a la iteración, agregando una constante distinta en cada iteración, y mezclando en general los bits minuciosamente. A dicha función se le conoce como ***función de hash criptográfica***.

Vale aclarar que, dependiendo de la bibliografía consultada, función de hashing puede ser considerada como un tipo específico de función de una vía, pero con características adicionales.



En esencia, la función hash, convierte cualquier dato digital en una cadena de caracteres de longitud fija (el "hash"). Esta cadena actúa como una "huella digital" única de los datos originales, y se utiliza para asegurar la integridad y autenticidad de la información, además de otras aplicaciones criptográficas.

Criterio	Características
¿Qué es?	Una función matemática fácil de calcular en un sentido, pero extremadamente difícil (inviable) de revertir.
¿Cómo actúa?	Manipula los bits de formas muy complejas para que sea impracticable volver al valor original.
Su resultado	Convierte cualquier dato digital (sin importar su tamaño) en una cadena de caracteres de longitud fija.
Su aplicación	Funciona como una "huella digital" única para asegurar la integridad de la información y almacenar datos de forma segura.

Firmas digitales.

Con frecuencia es necesario firmar un documento en forma digital. Por ejemplo, supongamos que un cliente pide a su banco que compre ciertas acciones, para lo cual le envía un mensaje de correo electrónico. Una hora después de haber enviado y ejecutado la orden, las acciones se desploman. Ahora el cliente niega incluso haber enviado el correo electrónico. Desde luego que el banco puede presentar el correo electrónico, pero el cliente puede afirmar que el banco lo falsificó para poder obtener una comisión.

Las firmas digitales hacen que sea posible firmar correos electrónicos y otros documentos digitales, de tal forma que el emisor no los pueda repudiar después. Una manera común es pasar primero el documento a través de un algoritmo de ***hashing criptográfico*** de una vía, que sea muy difícil de invertir.

Por lo general, la función de hashing produce un resultado de longitud fija, independiente del tamaño del documento original. Funciones populares de hashing fueron: ***MD5*** (Algoritmo de firma de mensajes 5), el cual produce un resultado de 16 bytes (Rivest, 1992) y ***SHA-1*** (Algoritmo de hash seguro), que produce un resultado de 20 bytes (NIST, 1995).

Versiones más recientes de SHA-1 son SHA-256 y SHA-512, que producen resultados de 32 y 64 bytes, respectivamente.

Algoritmo	Tamaño	Estado Actual
MD5	16 bytes	**Inseguro y vulnerable**
SHA-1	20 bytes	**Obsoleto e inseguro**
SHA-256	32 bytes	**Seguro (Estándar actual)**
SHA-512	64 bytes	**Muy seguro (Rendimiento alto)**

El siguiente paso asume que se está utilizando la criptografía de clave pública como vimos antes. Entonces, el propietario del documento aplica su clave privada al hash para obtener *D(hash)*. Este valor, conocido como ***bloque de firma*** (que es el hash cifrado con la clave privada), se adjunta al documento y se envía al receptor, como se muestra en la figura siguiente.

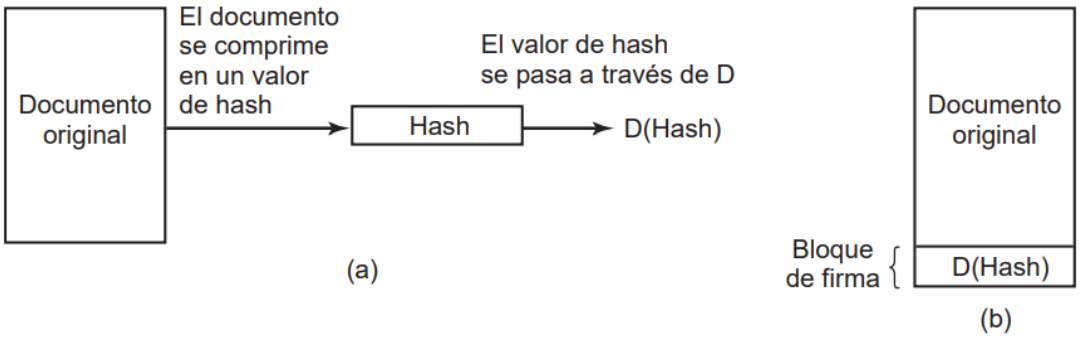


Ilustración 3 - (a) Cálculo de un bloque de firma. (b) Lo que obtiene el receptor.

Al proceso de aplicar *D* al hash se le conoce algunas veces como descifrar el hash, pero en realidad no es un descifrado debido a que el hash no se ha cifrado. Es sólo una transformación matemática aplicada al hash.

Cuando llegan el documento con su el bloque de firma, el receptor calcula primero el hash del documento mediante el uso de MD5 o SHA, según lo que hayan acordado de antemano. Después, el receptor aplica la clave pública del emisor al bloque de firma para obtener *E(D(hash))*.

En efecto, "cifra" el hash descifrado, lo cancela y obtiene el hash de vuelta. Si el hash calculado no coincide con el hash del bloque de firma, significa que se alteró el documento, el bloque de firma o ambos (o que se modificaron por accidente). El valor de este esquema es que aplica la criptografía de clave pública (lenta) sólo a una pieza de datos relativamente pequeña: el hash.

Para utilizar este esquema de firma, el receptor debe conocer la clave pública del emisor. Algunos usuarios publican su clave pública en su página Web. Otros no lo hacen debido al temor de que un intruso entre a su sistema y altere su clave sin que se enteren. Para ellos se requiere un mecanismo alternativo para distribuir claves públicas.

Un método común es que los emisores de un mensaje adjunten al mismo un certificado, el cual contiene el nombre del usuario y la clave pública, y está firmado digitalmente por una tercera parte de confianza. Una vez que el usuario haya adquirido la clave pública de la tercera parte de confianza, puede aceptar certificados de todos los emisores que utilicen esta tercera parte de confianza para generar sus certificados. A un tercero de confianza que firma certificados se le conoce como ***CA Certification Authority*** (Autoridad de certificación).

Sin embargo, para que un usuario pueda verificar un certificado firmado por una CA, necesita su clave pública. En general, se requiere todo un esquema para administrar claves públicas, conocido como PKI Public Key Infrastructure (Infraestructura de claves públicas).

Para los navegadores Web, el problema se resuelve ad hoc: todos los navegadores tienen ya cargadas las claves públicas populares.

1. El Emisor (Envío):

- Toma el documento y calcula su "huella digital" (su hash) con un algoritmo como SHA-256.
- Cifra esa huella digital usando su **clave privada**. El resultado es el **bloque de firma**.
- Envía por la red el documento junto con el bloque de firma.

2. El Receptor (Verificación):

- Recibe el documento y el bloque de firma.
- Por un lado, le calcula el hash al documento que le llegó.
- Por el otro lado, descifra el bloque de firma usando la **clave pública** del emisor para ver el hash original.

3. El Resultado:

- Compara los dos hashes. Si son idénticos, se asegura que el documento no cambió en el camino (**integridad**) y que realmente lo envió el dueño de la clave (**autenticidad**).

Módulo de plataforma confiable (TPM).

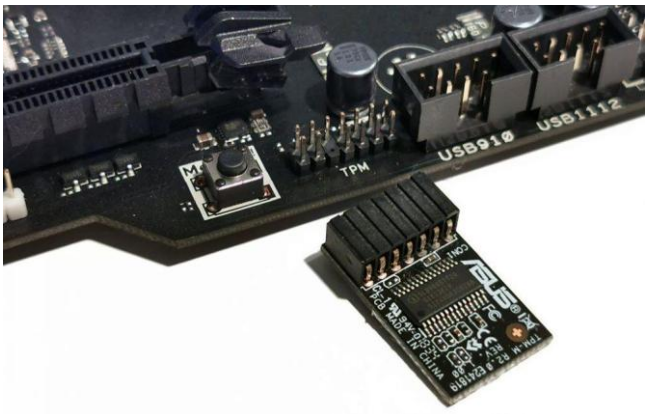
Toda la criptografía requiere claves. Si se comprometen las claves, también se compromete toda la seguridad que se basa en ellas. Por lo tanto, es esencial almacenar las claves de una forma segura.

La industria ha propuesto utilizar un chip llamado ***TPM*** (Trusted Platform Modules, Módulos de Plataforma Confiables), un ***criptoprocesador*** que contiene almacenamiento no volátil para guardar las claves.

El TPM puede realizar operaciones criptográficas como cifrar bloques de texto simple o descifrar bloques de texto cifrado en la memoria principal. También puede verificar firmas digitales. Al realizar estas operaciones en un hardware especializado, se pueden realizar con más velocidad y es probable que se utilicen con más frecuencia.

Algunas computadoras ya tienen chips TPM y es muy probable que la mayoría de las computadoras lo vayan a incluir en el futuro.

El TPM genera controversia, ya que hay distintas partes que tienen ideas diferentes acerca de quién va a controlar el TPM y de quién se va a proteger.



Microsoft es un gran defensor de ese concepto y ha desarrollado una serie de tecnologías para utilizarlo, incluyendo Palladium, NGSCB y BitLocker. En su punto de vista, el sistema operativo controla el TPM para evitar que se ejecute el software no autorizado. El “software no autorizado” podría ser software pirata (es decir, copias ilegales) o sólo software que el sistema operativo no autorice.

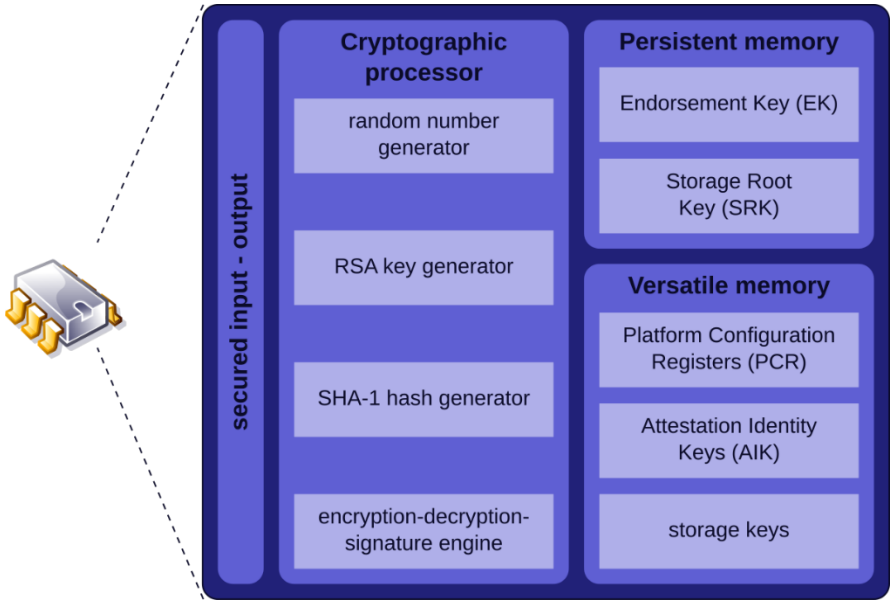
Si el TPM está involucrado en el proceso de arranque, podría iniciar sólo los sistemas operativos firmados por el fabricante mediante una clave secreta colocada dentro del TPM, que se divulgue sólo a ciertos distribuidores de sistemas operativos (como Microsoft).

Por ejemplo, el ***Arranque Seguro*** (Secure Boot) es una función del firmware UEFI que se apoya en el TPM, verifica las firmas digitales de cada componente de arranque antes de que se cargue.

Por lo tanto, el TPM se podría utilizar para limitar las opciones de los usuarios en cuanto al software, para que sólo puedan seleccionar el software aprobado por el fabricante de computadoras.

Las industrias de la música y las películas también tienen mucho interés en el TPM, ya que se puede utilizar para evitar que pirateen su contenido.

El TPM no ayuda a que las computadoras sean más seguras contra los ataques externos. En realidad, se enfoca en el uso de la criptografía para evitar que los usuarios hagan algo que no esté aprobado de manera directa o indirecta por la entidad que controle el TPM.



Es importe aclara que, en la actualidad, el TPM no se limita únicamente a un chip físico soldado en la placa madre. Los sistemas operativos y los procesadores modernos evolucionaron para implementar esta tecnología de diferentes maneras.

Tipos de TPM	• **dTPM (Dedicado):** El chip físico tradicional en la placa madre. • **fTPM (Firmware):** Integrado dentro del procesador principal (AMD/Intel), el más común hoy. • **vTPM (Virtual):** Clonado por software para máquinas virtuales.

Referencias y bibliografía de la sección.

- Tanenbaum 9-2.

Mecanismos de protección.

Es más fácil obtener seguridad si hay un modelo claro de lo que se debe proteger y de a quién se le permite hacer qué cosa. En este campo se ha realizado una cantidad considerable de trabajo, por lo que sólo podemos arañar la superficie.

Dominios de protección.

Un sistema computacional contiene muchos "objetos" que necesitan protección. Estos objetos pueden ser de hardware (por ejemplo, CPUs, segmentos de memoria, unidades de disco o impresoras) o de software (por ejemplo, procesos, archivos, bases de datos o semáforos).

Cada objeto tiene un nombre único que se utiliza como referencia, y hay un conjunto finito de operaciones que los procesos pueden llevar a cabo con los objetos. Las operaciones ***read*** y ***write*** son apropiadas para un archivo; ***up*** y ***down*** se utilizan en un semáforo. Es obvio que se necesita una manera de prohibir que los procesos utilicen objetos sin autorización.

Además, mediante este mecanismo se debe poder restringir a los procesos para que sólo puedan utilizar un conjunto de las operaciones legales cuando sea necesario. Por ejemplo, el proceso *A* tal vez tenga permiso de leer, pero no de escribir en el archivo *F*.

Para poder analizar distintos mecanismos de protección, es conveniente introducir el concepto de un ***dominio***. Un dominio es un conjunto de pares (objeto, permisos). Cada par especifica un objeto y cierto subconjunto de las operaciones que se pueden realizar en él. Un permiso en este contexto indica el permiso para realizar una de las operaciones. A menudo, un dominio corresponde a un solo usuario y le indica lo que puede hacer y lo que no, pero un dominio también puede ser más general que sólo un usuario. Por ejemplo, los miembros de un equipo de programación que trabajan en cierto proyecto podrían pertenecer al mismo dominio, de manera que todos tengan acceso a los archivos del proyecto.

La manera en que se asignan los objetos a los dominios depende de los detalles sobre quién necesita saber qué. Sin embargo, uno de los conceptos básicos es el POLA (Principle of Least Authority, Principio de menor autoridad), o la necesidad de saber.

En general, la seguridad funciona mejor cuando cada dominio tiene los mínimos objetos y privilegios para realizar su trabajo; y no más. En la figura que tenemos a continuación se muestran tres dominios, los objetos en cada dominio y los permisos disponibles en cada objeto: R (lectura), W (escritura) y X (ejecución).

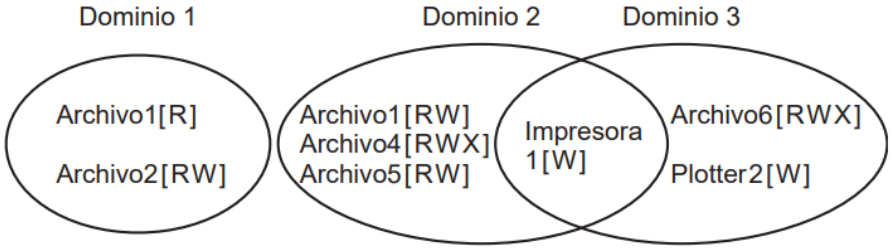


Ilustración 4 - Tres dominios de protección.

Observemos que *Impresora1* está en dos dominios al mismo tiempo, y tiene los mismos permisos en cada uno. *Archivo1* también está en dos dominios, pero tiene distintos permisos en cada uno.

En cada instante cada proceso se ejecuta en cierto dominio de protección. En otras palabras, hay cierta colección de objetos a los que puede acceder, y para cada objeto tiene cierto conjunto de permisos.

Los procesos también pueden cambiar de un dominio a otro durante la ejecución. Las reglas para cambiar de domino son muy dependientes del sistema. Para que la idea sobre un dominio de protección sea más concreta, vamos a analizar a UNIX (incluyendo a Linux, FreeBSD y sus amigos). En UNIX, el dominio de un proceso se define mediante su UID y su GID. Cuando un usuario inicia sesión, su shell recibe en el archivo de contraseñas el UID y el GID contenidos en su entrada, y todos sus hijos los heredan.

Dada cualquier combinación (UID, GID), es posible crear una lista completa de todos los objetos (archivos, incluyendo los dispositivos de E/S representados por archivos especiales, etc.) a los que se puede acceder, y si se pueden utilizar para leer, escribir o ejecutar. Dos procesos con la misma combinación (UID, GID) tendrán acceso al mismo conjunto exacto de objetos. Los procesos con distintos valores (UID, GID) tendrán acceso a un conjunto distinto de archivos, aunque se pueden traslapar en forma considerable.

Además, cada proceso en UNIX tiene dos mitades: la parte del usuario y la parte del kernel. Cuando el proceso realiza una llamada al sistema, cambia de la parte del usuario a la parte del kernel. Esta parte tiene acceso a un conjunto distinto de objetos que el de la parte del usuario. Por ejemplo, el kernel puede acceder a todas las páginas en la memoria física, todo el disco y todos los demás recursos protegidos. Por ende, una llamada al sistema produce un cambio de dominio.

Una cuestión importante es la forma en que el sistema lleva el registro de los dominios y qué objetos pertenecen a cada uno de ellos. En concepto, podemos por lo menos imaginar una gran matriz donde las filas son los dominios y las columnas los objetos. Cada cuadro lista los permisos (si los hay) que contiene el dominio para el objeto. En la figura siguiente se muestra la matriz para la figura anterior.

	Objeto							
	Archivo1	Archivo2	Archivo3	Archivo4	Archivo5	Archivo6	Impresora1	Plotter2
Dominio 1	Lectura	Lectura Escritura						
2			Lectura	Lectura Escritura Ejecución	Lectura Escritura		Escritura	
3						Lectura Escritura Ejecución	Escritura	Escritura

Ilustración 5 - Una matriz de protección.

Dada esta matriz y el número de dominio actual, el sistema puede saber si se permite el acceso a un objeto dado de una manera específica, y desde cierto dominio.

Se puede incluir con facilidad el cambio de dominio en el modelo de la matriz, para lo cual sólo hay que tener en cuenta que un dominio es en sí un objeto. La figura siguiente muestra la matriz de la figura anterior de nuevo, sólo que ahora con los tres dominios como objetos.

	Objeto										
	Archivo1	Archivo2	Archivo3	Archivo4	Archivo5	Archivo6	Impresora1	Plotter2	Dominio1	Dominio2	Dominio3
Dominio 1	Lectura	Lectura Escritura								Enter	
2			Lectura	Lectura Escritura Ejecución	Lectura Escritura		Escritura				
3						Lectura Escritura Ejecución	Escritura	Escritura			

Ilustración 6 - Una matriz de protección con dominios como objetos.

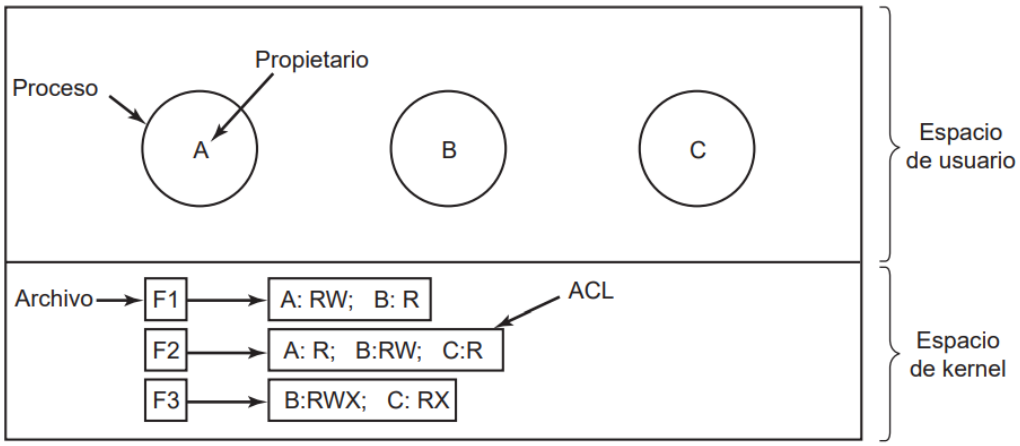
Los procesos en el dominio 1 se pueden cambiar al dominio 2, pero una vez ahí no pueden regresar. Esta situación modela la ejecución de un programa ***SETUID***[2] en UNIX. En este ejemplo no se permite ningún otro cambio de dominio.

Listas de control de acceso.

En la práctica raras veces se almacenan matrices de protección como la última que vimos en la sección anterior, ya que son extensas y escasas. La mayoría de los dominios no tienen acceso a la mayoría de los objetos, por lo que se desperdicia mucho espacio en disco al almacenar una matriz muy grande y en su mayor parte vacía. Sin embargo, hay dos métodos prácticos: almacenar la matriz por filas o columnas, y después almacenar sólo los elementos no vacíos.

[2] SETUID es un archivo ejecutable en sistemas operativos tipo Unix (como Linux) que permite a un usuario ejecutar el programa con los permisos del propietario del archivo, no con los permisos del usuario que lo ejecuta. Esto es útil para tareas que requieren mayores privilegios, como ejecutar comandos que modifican el sistema, sin tener que iniciar sesión como superusuario (root).

Los dos métodos son en extremo diferentes. Analizaremos el método de almacenar la matriz por columna. Consiste en asociar con cada objeto una lista (ordenada) que contenga todos los dominios que pueden acceder al objeto, y la forma de hacerlo. A esta lista se le conoce como ***Lista de control de acceso*** (Access Control List, o ***ACL***) y se ilustra en la figura que tenemos a continuación.



Acá podemos ver tres procesos, cada uno de los cuales pertenece a un dominio distinto: A, B y C, y tres archivos F1, F2 y F3. Por simplicidad vamos a suponer que cada dominio corresponde a sólo un usuario; en este caso, los usuarios A, B y C.

En la literatura sobre seguridad, a los usuarios se les denomina con frecuencia sujetos o protagonistas, para contrastarlos con las cosas que tienen dueño: los objetos, como los archivos.

Cada archivo tiene una ACL asociada. El archivo F1 tiene dos entradas en su ACL (separadas por un punto y coma). La primera entrada indica que cualquier proceso que sea propiedad del usuario A puede leer y escribir en el archivo. La segunda entrada indica que cualquier proceso que sea propiedad del usuario B puede leer el archivo. Todos los demás accesos para estos usuarios y los accesos para otros usuarios están prohibidos.

Observemos que los permisos se otorgan por usuario y no por proceso. En cuanto a lo que el sistema de protección concierne, cualquier proceso que sea propiedad del usuario A puede leer y escribir en el archivo F1. No importa si hay un proceso o 100; lo que importa es el propietario y no la ID del proceso.

El archivo F2 tiene tres entradas en su ACL: A, B y C pueden leer el archivo, y además B puede escribir en él. No se permiten otros accesos. Parece ser que el archivo F3 es un programa ejecutable, ya que B y C pueden leerlo y ejecutarlo. B también puede escribir en él. Este ejemplo ilustra la forma más básica de protección con las ACLs.

A menudo se utilizan sistemas más sofisticados en la práctica. Para empezar, hasta ahora sólo hemos mostrado tres permisos: lectura, escritura y ejecución. Puede haber también permisos adicionales. Algunos de ellos pueden ser genéricos, es decir, que se apliquen a todos los objetos, y algunos pueden ser específicos para cada objeto.

Algunos ejemplos de permisos genéricos son destroy object (destruir objeto) y copy object (copiar objeto). Estos permisos se pueden aplicar en cualquier objeto, sin importar cuál sea su tipo.

Los permisos específicos para cada objeto pueden ser append message (adjuntar mensaje) para un objeto bandeja de correo y sort alphabetically (ordenar alfabéticamente) para un objeto directorio.

Hasta ahora, las entradas en la ACL han sido para usuarios individuales. Muchos sistemas integran el concepto de un ***grupo de usuarios***.

Los grupos tienen nombres y se pueden incluir en las ACLs. Hay dos variaciones posibles en la semántica de los grupos. En ciertos sistemas, cada proceso tiene un ID de usuario (UID) y un ID de grupo (GID). En dichos sistemas, una entrada en la ACL contiene entradas de la siguiente forma:

UID1, GID1: permisos1; UID2, GID2: permisos2; …

Bajo estas condiciones, cuando se hace una petición para acceder a un objeto, se realiza una comprobación mediante el uso del UID y el GUID del proceso que hizo la llamada. Si están presentes en la ACL, están disponibles los permisos listados. Si la combinación (UID, GID) no está en la lista, no se permite el acceso.

Al utilizar los grupos de esta forma se introduce en efecto el concepto de un ***rol***. Consideremos una instalación de computadoras en la que Tana es administradora del sistema, y por ende está en el grupo sysadm. Sin embargo, suponga que la empresa también tiene clubs para empleados y que Tana es miembro del club de colombófilos[3]. Los miembros del club pertenecen al grupo fanpichones y tienen acceso a las computadoras de la empresa para administrar su base de datos de pichones.

En la figura siguiente se muestra una porción de lo que podría ser la ACL.

Archivo	Lista de control de acceso
Password	Tana, sysadm: RW
Pichones_data	bill, fanpichones: RW; tana, fanpichones: RW; …

Ilustración 7 - Dos listas de control de acceso.

Si Tana trata de acceder a uno de estos archivos, el resultado depende del grupo al que se encuentre conectada en ese momento. Cuando inicie sesión, el sistema le puede pedir que seleccione cuál de sus grupos está utilizando, o podría inclusive tener distintos nombres de inicio de sesión y/o contraseñas para mantenerlos separados.

[3] Persona que se dedica a la cría y adiestramiento de palomas mensajeras.

El objetivo de este esquema es evitar que Tana acceda al archivo de contraseñas cuando esté utilizando el grupo de fanáticos de pichones. Sólo puede hacer eso cuando esté conectada como administradora del sistema.

En algunos casos, un usuario puede tener acceso a ciertos archivos sin importar qué grupo esté utilizando en ese momento. Para manejar este caso vamos a introducir el concepto de ***comodín***, que representa a todos los usuarios. Por ejemplo:

tana, *: RW

La entrada anterior para el archivo de contraseñas proporciona acceso a Tana sin importar en qué grupo se encuentre.

Otra posibilidad más es que, si un usuario pertenece a cualquiera de los grupos que tengan ciertos permisos de acceso, se permite el acceso. La ventaja aquí es que un usuario que pertenezca a varios grupos no tiene que especificar el grupo que va a usar al momento de iniciar sesión. Todos los grupos son válidos en todo momento.

Una desventaja de este método es que no hay tanto encapsulamiento: Tana puede editar el archivo de contraseñas durante una reunión con el club de pichones. Al utilizar grupos y comodines se introduce la posibilidad de bloquear de manera selectiva a un usuario específico, para evitar que acceda a un archivo. Por ejemplo:

virgil, *: (none); *, *: RW

La entrada anterior proporciona a todos, excepto a Virgil, el acceso de lectura y escritura al archivo. Esto funciona debido a que las entradas se exploran en orden, y se toma la primera que se aplique; las entradas subsiguientes ni siquiera se examinan.

Hay una coincidencia para Virgil en la primera entrada y se aplican los derechos de acceso encontrados, que en este caso son “ninguno” (“none”). La búsqueda termina en ese punto. El hecho de que el resto del mundo tenga acceso ni siquiera se considera.

La otra manera de lidiar con los grupos es no tener entradas en la ACL que consistan en pares (UID, GID), sino que cada entrada sea un UID o un GID. Por ejemplo, una entrada para el archivo *pichones_data* podría ser:

debbie: RW; phil: RW; fanpichones: RW

Lo cual significa que Debbie y Phil, y todos los miembros del grupo *fanpichones* tienen acceso de lectura y de escritura al archivo.

Algunas veces sucede que un usuario o un grupo tiene ciertos permisos con respecto a un archivo, y posteriormente el propietario del archivo desea revocar esos permisos. Con las listas de control de acceso es muy sencillo revocar un acceso que se haya otorgado antes. Todo lo que hay que hacer es editar la ACL para realizar el cambio.

No obstante, si la ACL se revisa sólo cuando se abre un archivo, es muy probable que el cambio tenga efecto únicamente en las futuras llamadas a open. Cualquier archivo que ya se encuentre abierto seguirá con los derechos que tenía cuando se abrió, aun si el usuario ya no está autorizado para utilizarlo.

Capacidades.

La otra manera de dividir la última matriz de protección es por filas. Al utilizar este método, a cada proceso se le asocia una lista de objetos que puede utilizar, junto con una indicación de las operaciones permitidas en cada objeto; en otras palabras, su dominio.

A esta lista se le conoce como ***lista de capacidades*** (o ***lista-C***) y a los elementos individuales que contiene se les conoce como ***capacidades***.

En la figura que tenemos a continuación se muestra un conjunto de tres procesos y sus listas de capacidades.

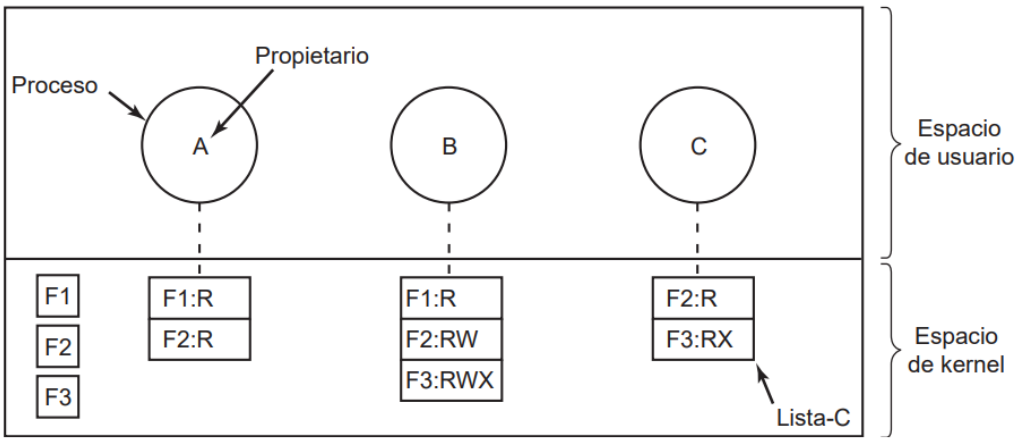


Ilustración 8 - Cuando se utilizan las capacidades, cada proceso tiene una lista de capacidades.

Cada capacidad otorga al propietario ciertos derechos sobre un objeto. Por ejemplo, el proceso que pertenece al usuario A puede leer los archivos F1 y F2.

Por lo general, una capacidad consiste en un identificador de archivo (o un objeto, en sentido más general) y un mapa de bits para los diversos permisos. En un sistema como UNIX el identificador de archivo probablemente sería el número de nodo-i. Las listas de capacidades son objetos en sí y otras listas de capacidades pueden apuntar a ellas, con lo cual se facilita la acción de compartir subdominios.

Además de los derechos específicos dependientes del objeto, como lectura y ejecución, las capacidades (que se protegen mediante el kernel y criptografía) por lo general tienen permisos genéricos que se pueden aplicar a todos los objetos. Algunos ejemplos de permisos genéricos son:

1. **Capacidad de copia:** crea una nueva capacidad para el mismo objeto.
2. **Copiar objeto:** crea un objeto duplicado con una nueva capacidad.
3. **Eliminar capacidad:** elimina una entrada de la lista-C; el objeto no se ve afectado.
4. **Destruir objeto:** elimina un objeto y una capacidad de manera permanente.

En resumen, las ***ACLs*** y las ***capacidades*** tienen propiedades que se complementan en cierta forma. Las capacidades son eficientes debido a que, si un proceso dice “Abrir el archivo al que apunta la capacidad 3”, no se necesita comprobación. Con las ACLs, tal vez se requiera una búsqueda (potencialmente extensa) en la ACL. Si no se aceptan los grupos, entonces para otorgar a todos el permiso de lectura para un archivo se requiere enumerar a todos los usuarios en la ACL.

Las capacidades también permiten encapsular un proceso con facilidad, mientras que las ACLs no.

Por otro lado, las ACLs permiten la revocación selectiva de los permisos, y las capacidades no. Por último, si se elimina un objeto, pero se dejan las capacidades, o si se eliminan las capacidades, pero se deja el objeto, surgen problemas. Las ACLs no sufren de este problema.

Sistemas confiables.

La única forma conocida de construir un sistema seguro es mantenerlo simple. Las características son enemigas de la seguridad. Los diseñadores de sistemas creen (pueden estar equivocados o no) que los usuarios quieren más características. Esto significa más complejidad, más código, más errores y más errores de seguridad.

He aquí dos ejemplos simples. Los primeros sistemas de correo electrónico enviaban mensajes en forma de texto ASCII. Eran completamente seguros. No hay nada que pueda hacer un mensaje entrante en código ASCII para dañar un sistema de cómputo. Después las personas tuvieron la idea de expandir el correo electrónico para incluir otros tipos de documentos. Por ejemplo, archivos de Word, que pueden contener programas en macros. Leer un documento de ese tipo significa ejecutar el programa de alguien más en su computadora. No importa cuántas cajas de arena se utilicen, ejecutar un programa externo en su computadora es mucho más peligroso que ver texto ASCII.

El segundo ejemplo es igual para las páginas Web. Cuando el servicio Web consistía en páginas de HTML pasivas, no imponía un problema grave de seguridad. Ahora que muchas páginas Web contienen programas que el usuario tiene que ejecutar para ver el contenido, aparece una fuga de seguridad tras otra. Tan pronto como se corrige una, otra toma su lugar

Hay varias organizaciones que piensan que es más importante una buena seguridad que características nuevas e ingeniosas, y los militares son el ejemplo principal.

Para construir un sistema seguro, hay que tener un modelo de seguridad en el núcleo del sistema operativo que sea lo bastante simple como para que los diseñadores puedan comprenderlo de verdad, y que resistan toda la presión de desviarse de este modelo para agregar nuevas características.

Base de cómputo confiable.

En el mundo de la seguridad, todos hablan con frecuencia sobre los sistemas confiables en vez de sistemas seguros. Éstos son sistemas que han declarado formalmente requerimientos de seguridad y cumplen con ellos. En el centro de todo sistema confiable hay una ***TCB*** (Trusted

Computing Base, Base de cómputo confiable) mínima que consiste en el hardware y software necesarios para cumplir con todas las reglas de seguridad.

Si la base de cómputo confiable funciona según las especificaciones, no se puede comprometer la seguridad el sistema, sin importar qué otra cosa esté mal. Por lo general, la TCB consiste en la mayoría del hardware (excepto los dispositivos de E/S que no afectan a la seguridad), una porción del kernel del sistema operativo, y la mayoría o todos los programas de usuario que tienen poder de superusuario (por ejemplo, los programas raíz ***SETUID*** en UNIX).

Las funciones del sistema operativo que deben formar parte de la TCB son: creación de procesos, cambio de procesos, administración del mapa de memoria y parte de la administración de archivos y de la E/S.

En un diseño seguro es frecuente que la TCB esté muy separada del resto del sistema operativo para poder minimizar su tamaño y verificar que sea correcto. ***El monitor de referencia*** es una parte importante de la TCB, como se muestra en la figura siguiente.

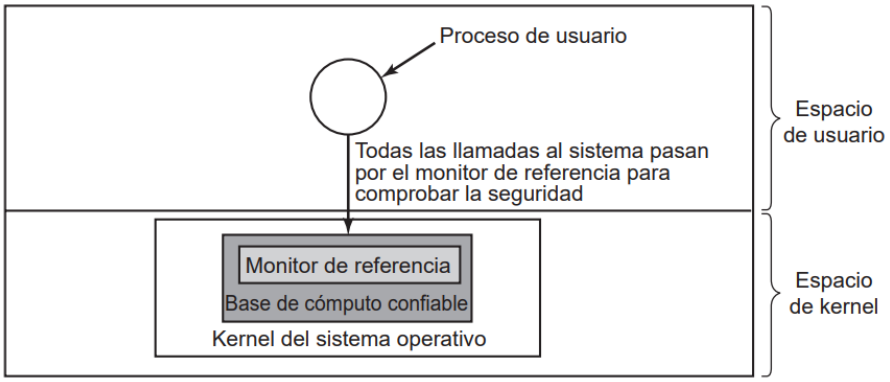


Ilustración 9 - Un monitor de referencia.

El monitor de referencia acepta todas las llamadas al sistema relacionadas con la seguridad (como abrir archivos), y decide si se deben procesar o no. Por ende, el monitor de referencia permite que todas las decisiones de seguridad se coloquen en un solo lugar, sin posibilidad de pasarlo por alto. La mayoría de los sistemas operativos están diseñados de esta forma, lo cual es una de las razones por las que son tan inseguros.

Uno de los objetivos de cierta investigación actual de seguridad es reducir la base de cómputo confiable, de millones de líneas de código a unas cuantas decenas de miles. En la figura siguiente vemos la estructura del sistema operativo MINIX 3, el cual es un sistema que se conforma a POSIX pero tiene una estructura radicalmente distinta a la de Linux o FreeBSD.

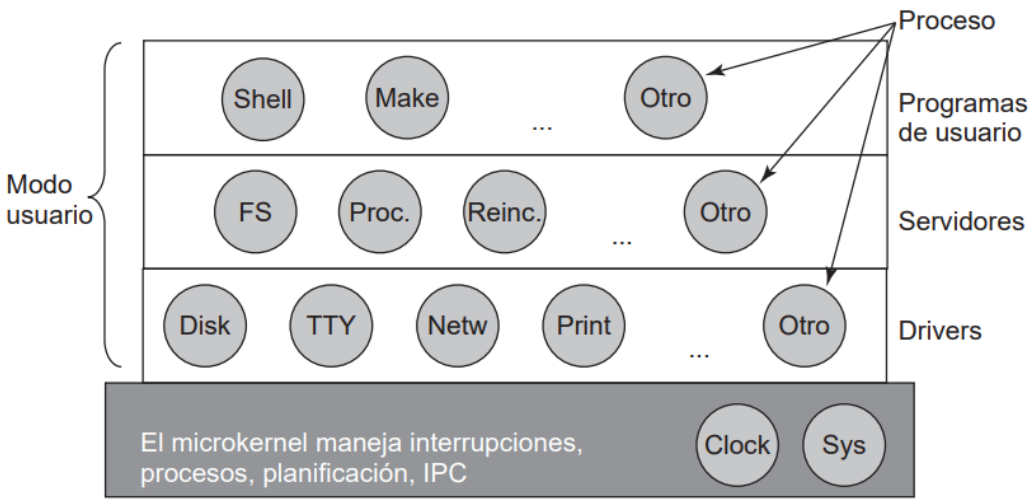


Ilustración 10 - Estructura del sistema MINIX 3.

Con MINIX 3, sólo se ejecutan aproximadamente 4000 líneas de código en el kernel. Todo lo demás se ejecuta como un conjunto de procesos de usuario. Algunos de estos procesos, como el sistema de archivos y el administrador de procesos, forman parte de la base de cómputo confiable debido a que pueden comprometer con facilidad la seguridad del sistema. Pero otras partes como el driver de impresora y el driver de audio no forman parte de la base de cómputo confiable, y no importa cuál sea su problema (incluso si un virus se apodera de ellos), no hay nada que se pueda hacer para comprometer la seguridad del sistema.

Referencias y bibliografía de la sección.

- Tanenbaum 9-3.

Autenticación, identificación y validación de usuarios.

Todo sistema computacional seguro debe requerir que todos los usuarios se autentiquen al momento de iniciar sesión. Después de todo, si el sistema operativo no puede estar seguro de quién es el usuario, tampoco puede saber a qué archivos y otros recursos puede acceder.

Aunque la autenticación puede parecer un tema trivial, es un poco más complicado de lo que se podría esperar.

Las primeras computadoras como la ENIAC no tenían un sistema operativo, mucho menos un procedimiento de inicio de sesión. Los posteriores sistemas mainframe de procesamiento por

lotes y tiempo compartido tenían por lo general un procedimiento de inicio de sesión para autenticar los trabajos y los usuarios.

Las primeras minicomputadoras (por ejemplo, PDP-1[4] y PDP-8) no tenían un procedimiento de inicio de sesión, pero con el esparcimiento de UNIX en la minicomputadora PDP-11, pasó a ser necesario tener uno.



Ilustración 11 - PDP-1 exhibida en el Computer History Museum de Mountain View, California.

Las primeras computadoras personales (por ejemplo, Apple II y la IBM PC original) no tenían un procedimiento de inicio de sesión, pero los sistemas operativos de computadora personal más sofisticados como Linux y Windows sí lo tienen (aunque los usuarios imprudentes lo pueden deshabilitar). Las máquinas en las LANs corporativas casi siempre tienen configurado un procedimiento de inicio de sesión, de manera que los usuarios no lo puedan evitar.

Por último, muchas personas hoy en día inician sesión (de manera indirecta) en computadoras remotas para realizar operaciones bancarias por Internet, compras electrónicas, descargar música y otras actividades comerciales. Todas estas actividades requieren de un inicio de sesión autenticado, por lo que una vez más la autenticación de los usuarios es un tema importante.

Habiendo determinado que la autenticación es a menudo importante, el siguiente paso es encontrar una buena manera de hacer que funcione. La mayoría de los métodos para autenticar usuarios cuando tratan de iniciar sesión se basan en uno de tres principios generales, que identifican:

1. Algo que el usuario conoce.

[4] PDP1, que significa "Procesador de Datos Programado - 1", fue la primera computadora producida por Digital Equipment Corporation (DEC) y se considera el primer miniordenador de la historia. Es un modelo histórico importante que se considera clave en la formación de la cultura hacker y en el desarrollo de los videojuegos.

2. Algo que el usuario tiene.
3. Algo que el usuario es.

Algunas veces se requieren dos de estos principios para una seguridad adicional. Estos producen distintos esquemas de autenticación con distintas complejidades y propiedades de seguridad.

En muchas ocasiones, las personas que desean ocasionar problemas en un sistema en particular primero tienen que iniciar sesión en ese sistema, para lo cual tienen que pasar por el procedimiento de autenticación que se utilice.

En la prensa popular, a estas personas se las conoce como hackers. Sin embargo, en el mundo de las computadoras la palabra “hacker” es un término honorario que se reserva para los grandes programadores.

Para diferenciar a los verdaderos hackers, utilizaremos el término en el sentido original y a las personas que tratan de irrumpir en los sistemas computacionales a los que no pertenecen les llamaremos ***crackers***.

Autenticación mediante el uso de contraseñas.

La forma más utilizada de autenticación es requerir que el usuario escriba un nombre de inicio de sesión y una contraseña. La protección mediante contraseñas es fácil de comprender y de implementar. La implementación más simple sólo mantiene una lista central de pares (nombre-iniciosesión, contraseña).

El nombre de inicio de sesión que se introduce se busca en la lista y la contraseña introducida se compara con la contraseña almacenada. Si coinciden, se permite al usuario que inicie sesión; en caso contrario, se rechaza.

Tal vez no haya necesidad de decir que mientras se escribe una contraseña en la computadora, ésta no debe mostrar los caracteres escritos, para evitar que alguien cercano al monitor los vea. En Windows, a medida que se escribe cada carácter aparece un asterisco. En UNIX no aparece ningún carácter a medida que se escribe la contraseña.

Estos esquemas tienen distintas propiedades. El esquema de Windows puede servir para que los usuarios distraídos vean fácilmente cuántos caracteres han escrito en cierto momento, pero esto también divulga la longitud de la contraseña a los fisgones. Desde la perspectiva de seguridad, el silencio es oro.

Otra área en la que hay implicaciones graves si no se realiza bien el trabajo es la que se muestra en la figura siguiente.

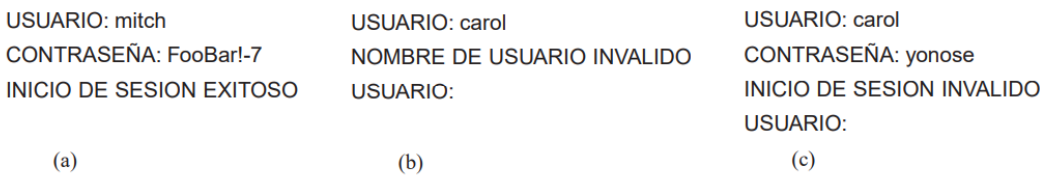


Ilustración 12 - (a) Un inicio de sesión exitoso. (b) Se rechaza el inicio de sesión después de introducir un nombre. (c) Se rechaza el inicio de sesión después de escribir el nombre y la contraseña

En la parte *a* se muestra un inicio de sesión exitoso, con la salida del sistema en mayúsculas y la entrada del usuario en minúsculas. En la parte *b* se muestra un intento fallido de un cracker por iniciar sesión en el Sistema A. En parte *c* se muestra un intento fallido de un cracker por iniciar sesión en el Sistema B.

En la parte *b* de la figura, el sistema se queja tan pronto como ve un nombre de inicio de sesión inválido. Esto es un error, ya que permite al cracker seguir probando nombres de inicio de sesión hasta que encuentra uno válido.

En la parte *c*, al cracker siempre se le pide una contraseña y no recibe ninguna pista sobre si el nombre de inicio de sesión es válido o no. Todo lo que aprende es que la combinación entre el nombre de inicio de sesión y la contraseña es incorrecta.

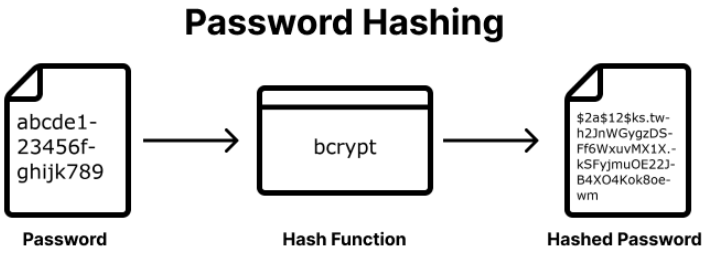
La mayoría de los BIOS permiten que el usuario proteja el programa de configuración con contraseña, de manera que sólo el propietario pueda cambiar la secuencia de arranque.

Seguridad de contraseñas de UNIX.

Algunos sistemas operativos (antiguos) mantienen el archivo de contraseñas en el disco en un formato no cifrado, pero protegido mediante los mecanismos usuales de protección del sistema.

Es muy peligroso tener todas las contraseñas en un archivo en el disco y en un formato no cifrado, ya que muchas personas tienen acceso a este archivo, y con mucha frecuencia. Entre estas personas puede haber administradores del sistema, operadores de las máquinas, personal de mantenimiento, programadores, la gerencia y tal vez hasta algunas secretarias.

Hay una mejor solución en UNIX, que se utiliza de la siguiente manera. El programa de inicio de sesión pide al usuario que escriba su nombre y su contraseña. La contraseña se “cifra” de inmediato al utilizarla como una clave para cifrar un bloque fijo de datos. En efecto, se ejecuta una función de una vía (función de hash), con la contraseña como entrada y una función de la contraseña como salida.



En realidad, este proceso no es cifrado. El cifrado es un proceso de dos vías (reversible), mientras que lo que hace el sistema operativo es un proceso de una vía (irreversible).De todas formas es mucho más fácil considerarlo así.

Después, el programa de inicio de sesión lee el archivo de contraseñas, que es sólo una serie de líneas ASCII, una por cada usuario, hasta que encuentra la línea que contiene el nombre de inicio de sesión del usuario. Si la contraseña (cifrada) que contiene esta línea coincide con la contraseña cifrada que se acaba de calcular se permite el inicio de sesión, y en caso contrario se rechaza.

La ventaja de este esquema es que nadie, ni siquiera el superusuario, puede ver las contraseñas de los usuarios debido a que no se almacenan en formato descifrado en ninguna parte del sistema. Sin embargo, este esquema también se puede atacar, como se muestra a continuación.

Primero, el cracker crea un diccionario de palabras probables. Estas contraseñas se cifran mediante el algoritmo conocido, a gusto del cracker. No importa cuánto tiempo tarde este proceso, ya que se realiza antes del intento de entrar en el sistema. Ahora que está armado con una lista de pares (contraseña, contraseña cifrada), el cracker ataca. Lee el archivo de contraseñas (que tiene acceso público) y extrae todas las contraseñas cifradas. Luego las compara con las contraseñas cifradas de su lista.

Por cada coincidencia, ahora se sabe el nombre de inicio de sesión y la contraseña descifrada. Una secuencia de comandos de shell simple puede automatizar este proceso, de manera que se pueda llevar a cabo en una fracción de un segundo.

Una ejecución típica de la secuencia de comandos producirá docenas de contraseñas. Al reconocer la posibilidad de este ataque, se ha descripto una técnica que inutiliza el ataque casi por completo. Su idea es asociar a cada contraseña un número aleatorio de *n* bits, conocido como ***salt***. El número aleatorio se modifica cada vez que se cambia la contraseña. El número aleatorio se almacena en el archivo de contraseñas en formato descifrado, para que todos puedan leerlo.

En vez de almacenar sólo la contraseña cifrada en el archivo de contraseñas, primero se concatenan la contraseña y el número aleatorio y después se cifran juntos. Este resultado cifrado se almacena en el archivo de contraseña, como se muestra en la figura siguiente para un archivo de contraseñas con cinco usuarios: Robbie, Tony, Laura, Mark y Deborah.

Bobbie, 4238, e(Perro, 4238)
Tony, 2918, e(6%%TaeFF, 2918)
Laura, 6902, e(Shakespeare, 6902)
Mark, 1694, e(XaB#Bwcz, 1694)
Deborah, 1092, e(LordByron, 1092)

Ilustración 13 - El uso de un salt para evitar que las contraseñas cifradas se calculen con anticipación.

Cada usuario tiene una línea en el archivo, con tres entradas separadas por comas: nombre de inicio de sesión, salt y contraseña cifrada + salt. La notación *e(Perro, 4238)* representa el resultado de concatenar la contraseña de Bobbie (Perro) con su salt asignado al azar (4238), y después ejecutarlo a través de la función de cifrado, *e*. El resultado de ese cifrado se almacena como el tercer campo de la entrada de Bobbie.

Ahora considere las implicaciones para un cracker que desea crear una lista de contraseñas probables, cifrarlas y guardar los resultados en un archivo ordenado f, de manera que se pueda buscar con facilidad cualquier contraseña cifrada. Si un intruso sospecha que Perro podría ser una contraseña, ya no basta con sólo cifrar Perro y colocar el resultado en f. Tiene que cifrar 2 a la *n* cadenas, como Perro0000, Perro0001, Perro0002 y así en lo sucesivo, e introducirlas todas en *f*.

Esta técnica incrementa el tamaño de *f* por 2 a la *n*. UNIX utiliza este método con *n* = 12.

Para obtener una seguridad adicional, algunas versiones modernas de UNIX hacen que el archivo de contraseñas sea ilegible, pero proporcionan un programa para buscar entradas en base a la petición, y agregan el suficiente retraso como para atrasar de manera considerable a cualquier atacante. En general, la combinación de usar un número salt con el archivo de contraseñas y hacerlo ilegible, excepto de manera indirecta (y lento), puede soportar la mayoría de los ataques.

Contraseñas de un solo uso.

La mayoría de los superusuarios exhortan a sus usuarios mortales para que cambien sus contraseñas una vez cada cierto tiempo. La mayoría de los usuarios hacen caso omiso.

Lo más extremoso es cambiar la contraseña en cada inicio de sesión; a estas contraseñas se les conoce como ***contraseñas de un solo uso***. Cuando se utilizan contraseñas de una sola vez, el usuario recibe un libro que contiene una lista de contraseñas. En cada inicio de sesión se utiliza la siguiente contraseña en la lista.

Si un intruso llega a descubrir una contraseña no le servirá de nada, ya que la próxima vez se debe utilizar una contraseña distinta.

Autenticación mediante el uso de un objeto físico.

El segundo método para autenticar a los usuarios es comprobar algún objeto físico que tengan, en vez de algo que sepan. Para este fin se han utilizado las llaves de puertas metálicas durante siglos. Hoy en día, el objeto físico que se utiliza con frecuencia es una tarjeta de plástico que se inserta en un lector asociado con la computadora.

Por lo general, el usuario no sólo debe insertar la tarjeta, sino que también debe escribir una contraseña para evitar que alguien utilice una tarjeta perdida o robada. Visto de esta forma, el uso de un ATM (Cajero automático) bancario empieza cuando el usuario inicia sesión en la computadora del banco a través de una terminal remota (la máquina ATM) mediante el uso de una tarjeta de plástico y una contraseña (en la actualidad es un código NIP de 4 dígitos en la mayoría de los países, pero esto sólo es para evitar el costo de tener que colocar un teclado completo en la máquina ATM).

Las tarjetas de plástico que contienen información vienen en dos variedades: tarjetas de tira magnética y tarjetas de chip.

Las tarjetas de tira magnética contienen aproximadamente 140 bytes de información escrita en una pieza de cinta magnética pegada en la parte posterior de la tarjeta. Una terminal puede leer esta información y enviarla a la computadora central. A menudo la información contiene la contraseña del usuario (por ejemplo, el código NIP) para que la terminal pueda realizar una comprobación de identidad, incluso aunque el enlace con la computadora principal esté desconectado. Por lo general, la contraseña se cifra mediante una clave que sólo el banco conoce.

Como una manera de identificar a los usuarios en general, las tarjetas de tira magnética son riesgosas debido a que el equipo para leerlas y escribir en ellas es económico y está disponible en muchas partes.

Las tarjetas de chip contienen un pequeño circuito integrado (chip). Estas tarjetas se pueden subdividir en dos categorías: tarjetas de valor almacenado y tarjetas inteligentes.

Las tarjetas de valor almacenado contienen una pequeña cantidad de memoria, en la que se utiliza la tecnología ROM para permitir que el valor se recuerde cuando se quite la tarjeta del lector y, por ende, se desconecte la energía. No hay CPU en la tarjeta, por lo que el valor almacenado se debe modificar mediante una CPU externa (en el lector).

En la actualidad una gran parte del trabajo sobre la seguridad se enfoca en las tarjetas inteligentes. Con el tiempo las tarjetas se están volviendo más inteligentes, pero están restringidas de varias formas, incluyendo la profundidad del chip (ya que está incrustado en la tarjeta), la anchura del chip (para que no se rompa cuando el usuario doble la tarjeta) y el costo (dependiendo del poder de la CPU, el tamaño de la memoria y la presencia o ausencia de un coprocesador criptográfico).

Las tarjetas inteligentes se pueden utilizar para guardar dinero al igual que las tarjetas de valor almacenado, pero con una mayor seguridad y universalidad. Las tarjetas se pueden cargar con dinero en una máquina ATM o en el hogar por medio del teléfono, utilizando un lector especial suministrado por el banco. Cuando se inserta en el lector de un comerciante, el usuario puede autorizar la tarjeta para deducir cierta cantidad de dinero de ella (al escribir SÍ), con lo cual la tarjeta envía un pequeño mensaje cifrado al comerciante. Después, el comerciante puede llevar el mensaje a un banco para cobrar el monto que se pagó.

La gran ventaja de las tarjetas inteligentes sobre las tarjetas de crédito o débito, por ejemplo, es que no necesitan una conexión en línea a un banco.

El concepto básico es simple: una tarjeta inteligente es una computadora pequeña a prueba de falsificaciones, que puede entablar una discusión (protocolo) con una computadora central para autenticar al usuario.

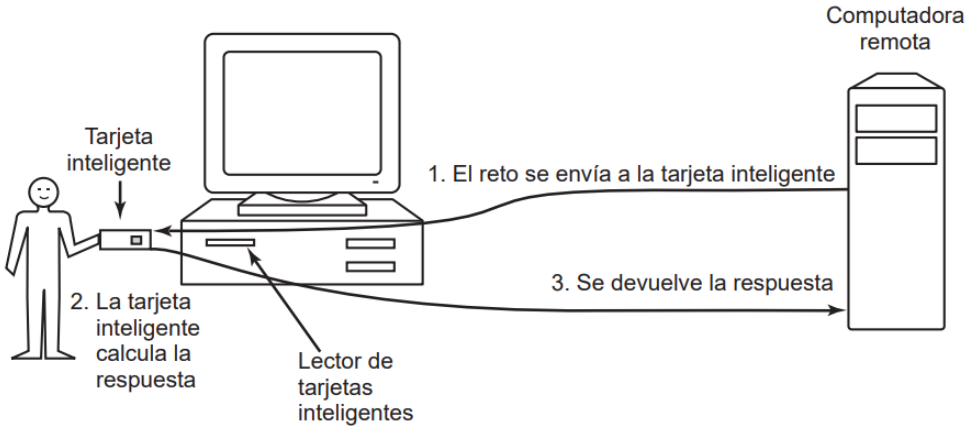


Ilustración 14 - Uso de una tarjeta inteligente para la autenticación.

Autenticación mediante biométrica.

El tercer método de autenticación mide las características físicas del usuario que son difíciles de falsificar. A estas características se les conoce como biométricas. Por ejemplo, un lector de huellas digitales o de voz conectado a la computadora podría verificar la identidad del usuario.

Un sistema biométrico común consta de dos partes: inscripción e identificación. Durante la inscripción se miden las características del usuario y se digitalizan los resultados. Las características importantes se extraen y almacenan en un registro asociado con el usuario. El registro se puede guardar en una base de datos central (por ejemplo, para iniciar sesión en una computadora remota) o se puede almacenar en una tarjeta inteligente que el usuario lleva consigo, y que inserta en un lector remoto (por ejemplo, una máquina ATM).

La otra parte es la identificación. El usuario aparece y proporciona un nombre de inicio de sesión. Después el sistema realiza de nuevo la medición. Si los nuevos valores coinciden con los que se muestrearon al momento de la inscripción, se acepta el inicio de sesión; en caso contrario se rechaza.

El nombre de inicio de sesión se necesita debido a que las mediciones nunca son exactas, por lo que es difícil indexarlas y después buscar con base en el índice. Además, dos personas podrían tener las mismas características, por lo que es más firme requerir que las características medidas coincidan con las de un usuario específico que sólo requerir que coincidan con las de cualquier usuario.

La característica elegida debe tener la suficiente variabilidad como para que el sistema pueda distinguir unas personas de otras sin error. Por ejemplo, el color del cabello no es un buen

indicador, ya que muchas personas comparten el mismo color. Además, la característica no debe variar con el tiempo, y en algunas personas el color del cabello no tiene esta propiedad.

De manera similar, la voz de una persona podría ser distinta si tiene un resfriado y un rostro podría verse diferente debido a una barba o por no traer maquillaje al momento de inscribirse.

Como las muestras posteriores nunca van a coincidir de manera exacta con los valores de la inscripción, los diseñadores del sistema tienen que decidir sobre la calidad de la coincidencia para poder aceptarla. En especial, tienen que decidir entre si es peor rechazar a un usuario legítimo de vez en cuando, o dejar que entre un impostor de vez en cuando.

Analicemos en forma breve algunas de las medidas biométricas que se utilizan en la actualidad. El análisis de la longitud de los dedos es sorprendentemente práctico. Cuando se utiliza, cada computadora tiene un dispositivo como el de la figura siguiente.

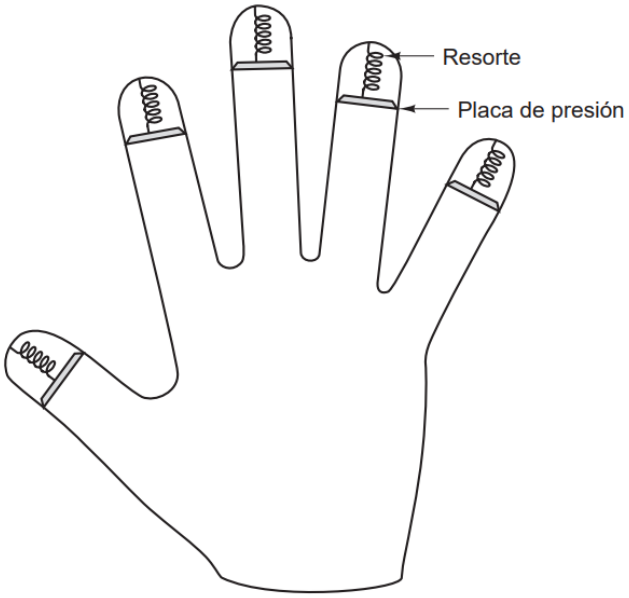


Ilustración 15 - Un dispositivo para medir la longitud de los dedos.

El usuario inserta su mano en él; a continuación, se mide la longitud de todos sus dedos y se comprueba en la base de datos.

Sin embargo, las mediciones de longitud de los dedos no son perfectas. El sistema puede ser atacado con moldes para manos hechos de yeso o de algún otro material, tal vez con dedos ajustables para permitir cierta experimentación.

Otra biométrica con uso comercial amplio es el reconocimiento del iris. No hay dos personas que tengan los mismos patrones (ni siquiera los gemelos idénticos), por lo que este método es tan bueno como el de las huellas digitales, y se automatiza con más facilidad. El sujeto sólo ve a través de una cámara (a una distancia de hasta 1 metro), la cual fotografía los ojos del sujeto y extrae ciertas características mediante lo que se conoce como una transformación con

ondículas de Gabor[5], y comprime los resultados a 256 bytes. Esa cadena se compara con el valor obtenido al momento de la inscripción.

Cualquier técnica que se base en imágenes está sujeta a la suplantación de identidad. Por ejemplo, una persona podría llegar al equipo (digamos, la cámara de una máquina ATM) usando lentes oscuros que estuvieran pegadas fotografías de los ojos de otra persona. Después de todo, si la cámara del ATM puede tomar una buena foto del iris a un metro, otras personas pueden hacerlo también, y a mayores distancias si utilizan teleobjetivos.

Por esta razón tal vez se necesiten contramedidas, como hacer que la cámara dispare un flash, no para fines de iluminación sino para ver si la pupila se contrae en respuesta, o para ver si el temido efecto de ojos rojos del fotógrafo amateur aparece en la imagen con flash, pero está ausente cuando no se utiliza flash. El aeropuerto de Amsterdam ha estado utilizando la tecnología de reconocimiento del iris desde 2001 para permitir que los viajeros frecuentes eviten la línea de inmigración normal.

El análisis de firmas es una técnica algo distinta. El usuario firma con una pluma especial conectada a la computadora, y ésta la compara con un espécimen conocido almacenado en línea, o en una tarjeta inteligente. Mejor aún es no comparar la firma, sino los movimientos de la pluma y la presión que se ejerce al escribir. Un buen falsificador tal vez pueda copiar la firma, pero no tendrá idea sobre el orden exacto en el que se hicieron los trazos, o con qué velocidad o presión se hicieron.

Referencias y bibliografía de la sección.

- Tanenbaum 9-4.

Defensas.

Analicemos algunas de las formas en que se pueden diseñar e implementar sistemas para incrementar su seguridad. Uno de los conceptos más importantes es la ***defensa en profundidad***.

La idea básica acá es que debemos tener varios niveles de seguridad, de manera que, si se viola uno de ellos, aún quedan otros niveles de defensa. Pensemos en una casa con una barda de hierro cerrada, alta y con picos a su alrededor, detectores de movimiento en el jardín, dos candados industriales en la puerta frontal y un sistema de alarma contra robos computarizado en su interior.

[5] La técnica wavelets de Gabor (Lee, 1996) es uno de los algoritmos básicos de procesamiento de imágenes y visión artificial para trabajos de extracción de características basados en la descripción de texturas mediante la descomposición de imágenes en diferentes orientaciones y escalas.

Aunque cada técnica es valiosa por sí sola, para robar la casa el ladrón tendría que vencerlas todas. Los sistemas computacionales con una seguridad apropiada son como esta casa, con varios niveles de seguridad.

En realidad, las defensas no son jerárquicas, pero empezaremos con las exteriores que son más generales, y avanzaremos hacia las más específicas.

Firewalls.

La habilidad de conectar cualquier computadora en cualquier parte con cualquier otra computadora en cualquier parte es una ventaja con claroscuros. Aunque hay mucho material valioso en la Web, una computadora conectada a Internet está expuesta a dos tipos de peligros: ***entrantes*** y ***salientes***.

Los peligros entrantes incluyen crackers que tratan de entrar a la computadora, así como virus, spyware y demás malware. Los peligros salientes incluyen información confidencial como los números de tarjetas de crédito, contraseñas, devoluciones de impuestos y todo tipo de información corporativa.

En consecuencia, se requieren mecanismos para mantener los bits “buenos” adentro y los bits “malos” afuera. Uno de estos métodos es el uso de un ***firewall***, que es tan sólo una adaptación moderna de ese antiguo recurso de seguridad medieval: cavar un foso profundo alrededor del castillo.

Este diseño obligaba a todos los que entraban o salían del castillo a pasar por un solo puente levadizo, en donde la policía de E/S podía inspeccionarlos. Con las redes es posible el mismo truco: una empresa puede tener muchas LANs conectadas en formas arbitrarias, pero todo el tráfico entrante y saliente de la empresa pasa de manera obligatoria a través de un puente levadizo electrónico, el firewall.

Hay dos variedades básicas de firewall: de hardware y de software. Por lo general, las empresas que deben proteger sus LANs optan por los firewalls de hardware; los individuos en su hogar eligen con frecuencia los firewalls de software.

Analicemos los ***firewalls de hardware***. En la figura que se muestra a continuación se ilustra un firewall de hardware genérico.

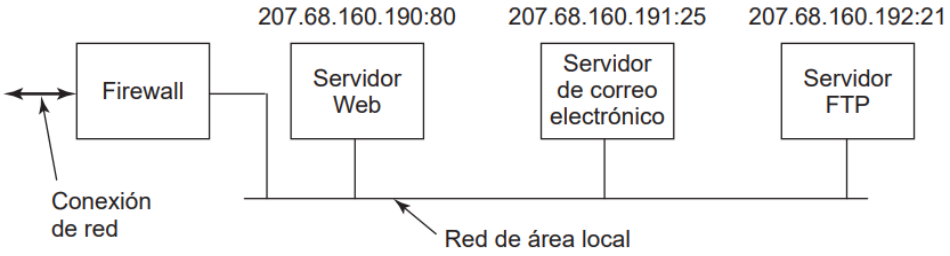


Ilustración 16 - Una vista simplificada de un firewall de hardware que protege a una LAN con tres computadoras.

Acá, la conexión (cable o fibra óptica) del proveedor de red se conecta al firewall, el cual se conecta a la LAN. No pueden entrar paquetes a la LAN ni salir de ella sin que el firewall lo apruebe. En la práctica, los firewalls se combinan a menudo con enrutadores, cajas de resolución de direcciones, sistemas de detección de intrusos y otras cosas, pero nuestro enfoque aquí será sobre la funcionalidad del firewall.

Los firewalls se configuran con reglas que describen lo que puede entrar y lo que puede salir. El propietario del firewall puede cambiar las reglas, por lo común a través de una interfaz Web (la mayoría de los firewalls tienen un mini servidor Web integrado, para permitir esto).

En el tipo de firewall más simple, el firewall sin estado, se inspecciona el encabezado de cada paquete que pasa por él y se toma la decisión de aceptar o rechazar el paquete, sólo con base en la información del encabezado y las reglas del firewall. La información en el encabezado del paquete incluye las direcciones IP de origen y destino, los puertos de origen y destino, el tipo de servicio y el protocolo. Hay otros campos disponibles, pero ocurren raras veces en las reglas.

En el ejemplo de la figura anterior podemos ver tres servidores, cada uno con una dirección IP única de la forma 207.68.160.x, en donde x es 190, 191 y 192 respectivamente. Estas son las direcciones a las que se deben enviar los paquetes para que lleguen a estos servidores. Los paquetes entrantes también contienen un número de puerto de 16 bits, que especifica cuál proceso en la máquina debe recibir el paquete (un proceso puede escuchar en un puerto el tráfico entrante).

Algunos puertos tienen servicios asociados. En especial, el puerto 80 se utiliza para Web, el puerto 25 para el correo electrónico y el puerto 21 para el servicio FTP (transferencia de archivos), pero la mayoría de los demás puertos están disponibles para los servicios definidos por el usuario.

Bajo estas condiciones, el firewall se podría configurar de la siguiente forma:

Dirección IP	Puerto	Acción
207.68.160.190	80	Aceptar
207.68.160.191	25	Aceptar
207.68.160.192	21	Aceptar
*	*	Rechazar

Estas reglas permiten a los paquetes ir a la máquina 207.68.160.190, pero sólo si están dirigidos para el puerto 80; todos los demás puertos en esta máquina están prohibidos y el firewall descartará en silencio los paquetes que se envíen a ellos.

De manera similar, los paquetes pueden ir a los otros dos servidores si están dirigidos a los puertos 25 y 21, respectivamente. El resto del tráfico se descarta.

Con este conjunto de reglas es difícil que un atacante obtenga acceso a la LAN, excepto por los tres servicios públicos que se ofrecen.

A pesar del firewall, aún es posible atacar la LAN. Por ejemplo, si el servidor Web es Apache[6] y el cracker descubre un error en Apache que puede explotar, podría enviar un URL muy largo a

[6] Apache Server, comúnmente conocido como Apache, es un servidor web de código abierto, gratuito y multiplataforma que se utiliza para servir contenido web a través de Internet. Es uno de los servidores

la dirección 207.68.160.190 en el puerto 80 y forzar un desbordamiento de búfer, con lo cual controlaría una de las máquinas dentro del firewall y la podría utilizar para lanzar un ataque sobre las otras máquinas en la LAN.

Otro ataque potencial es escribir y publicar un juego multijugador, y lograr que tenga una amplia aceptación. El software del juego necesita cierto puerto para conectarse con otros jugadores, por lo que el diseñador del juego podría seleccionar uno (por ejemplo, 9876) e indicar a los jugadores que cambien la configuración de su firewall para permitir el tráfico entrante y saliente en este puerto. Las personas que abran este puerto estarán sujetas a sufrir ataques, lo cual podría ser sencillo si el juego contiene un caballo de Troya que acepte ciertos comandos remotos y los ejecute.

Pero incluso si el juego es legítimo, podría contener errores que se puedan explotar. Entre más puertos estén abiertos, mayor será la probabilidad de que un ataque sea exitoso. Cada hoyo incrementa la posibilidad de que se infiltre un ataque.

Además de los firewalls sin estado, también hay firewalls de estado, que mantienen el registro de las conexiones y el estado en el que se encuentran. Estos firewalls son mejores para vencer ciertos tipos de ataques, en especial los que están relacionados con el establecimiento de conexiones.

Hay otros tipos de firewalls que implementan un IDS (Intrusion Detection System, Sistema de detección de intrusos), donde el firewall no sólo inspecciona los encabezados de los paquetes sino también su contenido, en busca de material sospechoso.

Los ***firewalls de software*** (a los que también se les conoce como firewalls personales) hacen lo mismo que los firewalls de hardware, pero en software. Son filtros que se conectan al código de red dentro del kernel del sistema operativo y filtran los paquetes de la misma forma en que lo hace el firewall de hardware.

Los antivirus y las técnicas anti-antivirus.

Los firewalls tratan de mantener a los intrusos fuera de la computadora, pero pueden fallar de diversas formas. En ese caso, la siguiente línea de defensa está compuesta por los programas antimalware, que se conocen comúnmente como programas antivirus, aunque muchos de ellos también combaten los gusanos y el spyware.

Los virus tratan de ocultarse y los usuarios tratan de encontrarlos. En este sentido los virus son como los rootkits, excepto que la mayoría de los escritores de virus se enfocan en el rápido esparcimiento del virus, en vez de jugar a las escondidas como los rootkits.

Ahora analicemos algunas de las técnicas utilizadas por el software antivirus y la forma en que el escritor de virus, responde a ellos.

web más populares y, en esencia, actúa como un intermediario entre los servidores web y los usuarios que desean acceder a ellos.

Exploradores de virus.

Sin duda, el usuario promedio no va a poder encontrar muchos de los virus que hacen su mejor esfuerzo por ocultarse, por lo que se ha desarrollado un mercado para el software antivirus.

Las empresas de software antivirus tienen laboratorios en los que científicos dedicados trabajan largas horas para rastrear y comprender los nuevos virus. El primer paso es hacer que el virus infecte un programa que no hace nada, conocido como archivo señuelo (goat file), para obtener una copia del virus en su forma más pura. El siguiente paso es hacer un listado exacto del código del virus e introducirlo en la base de datos de virus conocidos.

Las empresas compiten por el tamaño de sus bases de datos.

Una vez que se instala un programa antivirus en la máquina de un cliente, lo primero que hace es explorar todos los archivos ejecutables en el disco, en busca de cualquiera de los virus en la base de datos de virus conocidos.

Si el usuario tiene 10,000 archivos y la base de datos tiene 10,000 virus, sin duda se requiere cierta programación inteligente para agilizar el proceso. Como surgen variantes menores de virus conocidos todo el tiempo, se necesita una búsqueda difusa[7] para asegurar que un cambio de 3 bytes en un virus no evite su detección.

Sin embargo, las búsquedas difusas no sólo son más lentas que las exactas, sino que pueden activar falsas alarmas (falso positivo); es decir, advertencias sobre archivos legítimos que sólo contienen cierto código algo similar a un virus reportado en Pakistán hace 7 años

Entre más virus haya en la base de datos y más amplio sea el criterio para declarar una ocurrencia, habrá más falsas alarmas. Si hay demasiadas, el disgustado usuario se rendirá. Pero si el explorador de virus insiste en una ocurrencia muy cercana, puede pasar por alto algunos virus modificados.

Obtener una detección acertada es un balance delicado de heurística. En teoría, el laboratorio debería tratar de identificar cierto código básico en el virus que no tenga probabilidades de cambiar, y utilizarlo como la firma del virus que se debe buscar. El hecho de que el disco se haya declarado libre de virus la semana pasada no significa que aún lo esté, por lo cual hay que ejecutar el explorador de virus con frecuencia.

Como la exploración es lenta, es más eficiente comprobar sólo los archivos que se han modificado desde la fecha de la última exploración. El problema es que un virus inteligente restablecerá la fecha de un archivo infectado a su fecha original, para evitar ser detectado. La respuesta del programa antivirus a esta acción es comprobar la fecha en que se modificó el directorio que contiene el archivo.

La respuesta del virus a esta acción es restablecer también la fecha del directorio. Este es el inicio del ***juego del gato y el ratón***.

Otra forma en que el programa antivirus puede detectar la infección de archivos es registrar y almacenar en el disco las longitudes de todos los archivos. Si un archivo creció en tamaño desde la última comprobación, podría estar infectado como se muestra en la figura siguiente (a y b).

[7] La búsqueda difusa es una técnica que permite encontrar coincidencias incluso cuando la consulta no es idéntica a los datos que se buscan.

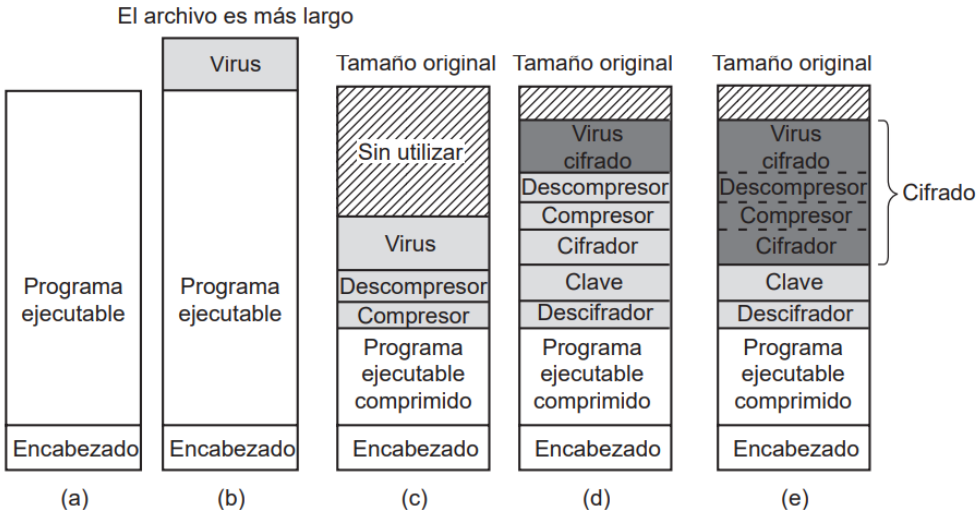


Ilustración 17 - (a) Un programa. (b) Un programa infectado. (c) Un programa infectado comprimido. (d) Un virus cifrado. (e) Un virus comprimido con el código de compresión cifrado.

Sin embargo, un virus inteligente puede evitar la detección al comprimir el programa y rellenar el espacio libre del archivo para que tenga su longitud original. Para que este esquema pueda funcionar, el virus debe contener procedimientos tanto de compresión como de descompresión, como se muestra en la parte *c* de la figura.

Otra forma en que el virus puede tratar de evitar la detección es asegurarse de que su representación en el disco no tenga la apariencia de su representación en la base de datos del software antivirus.

Una manera de lograr este objetivo es que se cifre a sí mismo con una clave distinta para cada archivo infectado. Antes de realizar una nueva copia, el virus genera una clave de cifrado aleatoria de 32 bits; por ejemplo, aplicando un XOR[8] al tiempo actual y el contenido de (por ejemplo) las palabras de memoria 72,008 y 319,992. Después aplica un XOR a su código con esta clave, palabra por palabra, para producir el virus cifrado que se almacena en el archivo infectado, como se muestra en la parte *d* de la figura anterior.

La clave se almacena en el archivo. Para fines de mantenerse en secreto no es ideal colocar la clave en el archivo, pero el objetivo aquí es frustrar al explorador de virus, no evitar que los científicos dedicados en el laboratorio antivirus apliquen ingeniería inversa al código.

Desde luego que, para ejecutarse, el virus tiene que descifrarse a sí mismo primero, por lo que también necesita una función de descifrado en el archivo. Este esquema no es perfecto debido a que los procedimientos de compresión, descompresión, cifrado y descifrado son los mismos en todas las copias, y el programa antivirus los puede utilizar como la firma del antivirus a buscar en el proceso de exploración.

[8] El cifrado XOR es una técnica de cifrado simple que utiliza la operación lógica XOR (OR exclusivo) para encriptar y desencriptar datos. Es una forma básica de cifrado simétrico donde la misma clave se usa para cifrar y descifrar el mensaje.

Es fácil ocultar los procedimientos de compresión, descompresión y cifrado: sólo se cifran junto con el resto del virus, como se muestra en la parte *e* de la figura.

Sin embargo, el código de descifrado no se puede cifrar, ya que se tiene que ejecutar en el hardware para descifrar el resto del virus, por lo que debe estar presente en texto simple. Los programas antivirus ya saben esto, por lo que buscan el procedimiento de descifrado.

Por supuesto, los creadores de virus no se quedarán conformes con este desenlace; el juego del gato y el ratón podría extenderse indefinidamente.

Hasta ahora sólo hemos hablado de tratar de reconocer virus en archivos ejecutables infectados. Además de esto, el explorador de virus tiene que comprobar el MBR, los sectores de arranque, la lista de sectores defectuosos, la memoria flash, la memoria CMOS y otras cosas más.

Supongamos que el virus en ejecución monitorea todas las llamadas al sistema. De esta forma puede detectar con facilidad que el programa antivirus está leyendo el sector de arranque (para comprobar si hay virus). Para frustrar al programa antivirus, el virus no realiza la llamada al sistema. En vez de ello, sólo devuelve el verdadero sector de arranque desde su lugar oculto en la lista de bloques defectuosos. También hace una anotación mental para volver a infectar todos los archivos cuando el explorador de virus termine.

Para evitar ser engañado por un virus, el programa antivirus podría hacer lecturas directas en el disco, pasando por alto al sistema operativo. Sin embargo, para ello se requiere tener drivers de dispositivos para los discos SATA, IDE y otros discos comunes, con lo cual el programa antivirus sería menos portátil y estaría sujeto a fallas en las computadoras con discos inusuales.

Comprobadores de integridad.

La comprobación de integridad es un método completamente distinto para detectar virus. Un programa antivirus que funciona de esta manera explora primero el disco duro en busca de virus. Una vez que está convencido de que el disco está limpio, calcula una suma de comprobación para cada archivo ejecutable.

El algoritmo de sumas de comprobación podría ser algo tan simple como tratar a todas las palabras en el texto del programa como enteros de 32 o 64 bits y sumarlas, pero también podría ser un hash criptográfico que sea casi imposible de invertir.

Después escribe en un archivo llamado ***sumacomp*** la lista de sumas de comprobación para todos los archivos relevantes en un directorio, en ese directorio. La próxima vez que se ejecuta, vuelve a calcular todas las sumas de comprobación y verifica que coincidan con lo que hay en el archivo sumacomp.

Un archivo infectado en la lista se podrá identificar de inmediato. El problema es que el creador del virus no va a estar de brazos cruzados ante esto. Puede escribir un virus que elimine el archivo de comprobación. O peor aún, puede escribir un virus que calcule la suma de comprobación del archivo infectado y reemplace la entrada anterior en el archivo de sumas de comprobación.

Para protegerse contra este tipo de comportamiento, el programa antivirus puede tratar de ocultar el archivo de sumas de comprobación, pero no es probable que esto vaya a funcionar debido a que el escritor del virus puede estudiar el programa antivirus con cuidado antes de escribir el virus.

Una mejor idea sería firmarlo digitalmente para que sea más fácil detectar su alteración. En teoría, la firma digital debe implicar el uso de una tarjeta inteligente con una clave almacenada en forma externa, que los programas no puedan alcanzar.

Comprobadores del comportamiento.

La comprobación del comportamiento es la tercera estrategia que utiliza el software antivirus. Con este método, el programa antivirus vive en memoria mientras que la computadora está funcionando, y atrapa por sí solo todas las llamadas al sistema.

La idea es que pueda entonces monitorear toda la actividad y trate de atrapar algo que se vea sospechoso. Por ejemplo, ningún programa normal debería tratar de sobrescribir el sector de arranque, por lo que es casi un hecho que un intento por hacer esto se debe a un virus.

De igual forma, cambiar la memoria flash es una actitud muy sospechosa. Pero también hay casos menos claros. Por ejemplo, sobrescribir un archivo ejecutable es algo peculiar (a menos que lo haga un compilador). Si el software antivirus detecta dicha escritura y emite una advertencia, se espera que el usuario sepa si tiene sentido sobrescribir un ejecutable en el contexto del trabajo actual.

De manera similar, el que Word sobrescriba un archivo .doc con un nuevo documento lleno de macros no es necesariamente el trabajo de un virus. En Windows, los programas se pueden separar de su archivo ejecutable y permanecer residentes en memoria mediante el uso de una llamada al sistema especial. Como dijimos antes, esto podría ser legítimo, pero de todas formas sería útil una advertencia.

Los virus no tienen que permanecer pasivos en espera de que un programa antivirus los elimine, como el ganado que se lleva a sacrificar. Pueden responder al ataque. Se puede llevar a cabo una batalla bastante acalorada si un virus residente en memoria y un antivirus se encuentran en la misma computadora.

Hace años había un juego llamado Core Wars en el que se enfrentaban dos programadores, y cada uno de ellos colocaba un programa en un espacio de direcciones vacío. Los programas tomaban turnos para sondear la memoria, y el objetivo del juego era localizar y borrar al oponente antes de que él hiciera lo mismo.

La confrontación entre virus y antivirus es algo similar, sólo que el campo de batalla es la máquina de un pobre usuario que en realidad no quiere que la confrontación ocurra ahí. Lo que es peor, el virus tiene una ventaja ya que su escritor puede averiguar muchos detalles sobre el programa antivirus con sólo comprar una copia.

Desde luego que cuando el virus se libera, el equipo del antivirus puede modificar su programa para obligar al escritor del virus a comprar una nueva copia.

Cómo evitar los virus.

Es mucho más fácil evitar los virus que tratar de rastrearlos después de que han infectado una computadora.

Por supuesto que es buena idea elegir un sistema operativo que ofrezca un alto nivel de seguridad, con un límite sólido entre el modo de kernel y el modo de usuario, y contraseñas separadas para cada usuario, además del administrador del sistema. Bajo estas condiciones, un virus que se escabulla de alguna forma no podrá infectar los archivos binarios del sistema.

En segundo lugar, se puede instalar sólo software legal que se compre de algún fabricante confiable. Aunque, esto no garantiza nada, sí ayuda mucho, ya que han ocurrido casos en que los empleados descontentos introducen virus en un producto de software comercial.

En tercer lugar, se puede comprar un buen paquete de software antivirus y usarlo según las indicaciones. Hay que asegurarse de obtener actualizaciones periódicas del fabricante.

En cuarto lugar, se pueden realizar copias frecuentes de los archivos clave en un medio externo, como pendrives. Es conveniente mantener varias generaciones de cada archivo en una serie de medios de respaldo. De esta forma, si el usuario descubre un virus, puede tener la oportunidad de restaurar los archivos al estado que tenían antes de ser infectados. No es de mucha ayuda restaurar el archivo infectado de ayer, pero la versión de la semana pasada tal vez si sea de ayuda.

En quinto y último lugar, deben resistir la tentación de descargar y ejecutar el nuevo software ostentoso y gratuito de una fuente desconocida. Tal vez haya una razón por la cual sea gratuito: que el fabricante desea que nuestra computadora se una a su ejército de zombies.

Aunque si usted tiene software de máquina virtual, es seguro ejecutar software desconocido dentro de ella. La industria también debería tomar en serio la amenaza de los virus y modificar algunas prácticas peligrosas.

Los sistemas operativos deben ser simples. Entre más características espectaculares tengan, habrá más hoyos de seguridad.

Firma de código.

Un método completamente distinto de impedir que el malware entre en la computadora es ejecutar sólo el software de distribuidores confiables que no tenga modificaciones.

Una de las cuestiones que surgen con mucha rapidez es la forma en que el usuario puede saber que el software realmente proviene del distribuidor correcto, y cómo puede saber que no se ha modificado desde que salió de la fábrica.

Esta cuestión es muy importante cuando se descarga software de tiendas en línea con una reputación desconocida, o al descargar controles activeX de los sitios Web. Si el control activeX proviene de una empresa de software reconocida, es poco probable que contenga un caballo de Troya, por ejemplo.

Un método que se utiliza con mucha frecuencia, para determinar el origen seguro del software es la firma digital. Si el usuario sólo ejecuta programas, complementos, drivers, controles

activeX y otros tipos de software escritos y firmados por fuentes confiables, las probabilidades de tener problemas son mucho menores.

La firma de código se basa en la criptografía de clave pública. Un distribuidor de software genera un par (clave pública, clave privada), y pone la primera clave a disposición del público, pero guarda la segunda clave con recelo.

Para firmar una pieza de software, el distribuidor primero calcula una función de hash del código para obtener un número de 128, 160 o 256 bits, dependiendo de si se utiliza MD5, SHA-1 o SHA-256. Después, para firmar el valor de hash lo cifra con su clave privada. Esta firma acompaña al software a cualquier lado que vaya. Cuando el usuario recibe el software, se le aplica la función de hash y se guarda el resultado. Después descifra la firma que lo acompaña mediante la clave pública del distribuidor y compara el resultado que tiene el distribuidor de la función de hash con el resultado que acaba de calcular. Si coinciden, el código se acepta como genuino. En caso contrario, se rechaza como una falsificación.

Debido a los cálculos matemáticos involucrados, es muy difícil que alguien pueda alterar el software de tal forma que su función de hash coincida con la función de hash que se obtiene al descifrar la firma genuina. Es igual de difícil generar una nueva firma falsa que coincida sin tener la clave privada. En la figura siguiente se ilustran los procesos de firma y verificación.

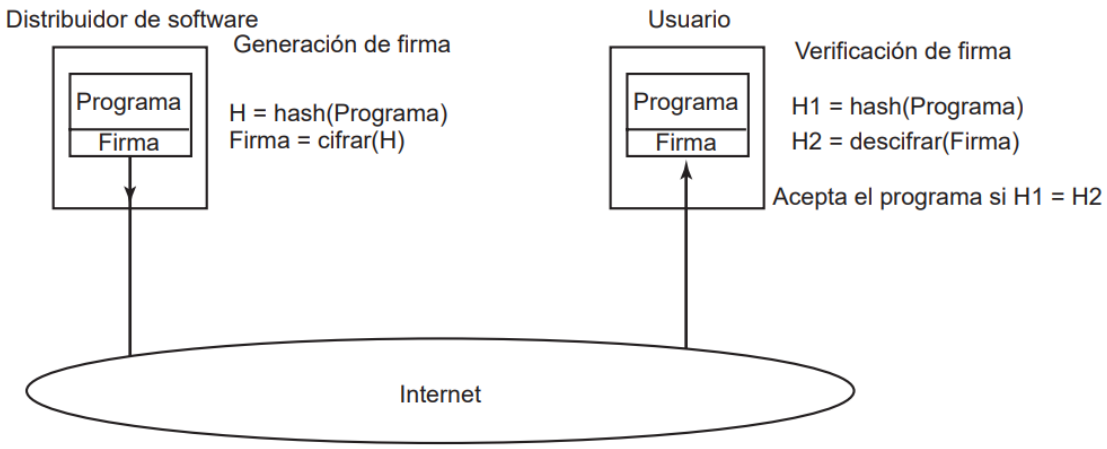


Ilustración 18 - Cómo funciona la firma de código.

Las páginas Web pueden contener código como los controles activeX, pero también código en diversos lenguajes de secuencias de comandos. A menudo este código está firmado, en cuyo caso el navegador examina la firma de manera automática. Claro que para verificarla el navegador necesita la clave pública del distribuidor de software, que por lo general se incluye con el código junto con un certificado firmado por una CA que responde por la autenticidad de la clave pública. Si el navegador ya tiene guardada la clave pública de la CA, puede verificar el certificado por su cuenta. Si el certificado está firmado por una CA que el navegador desconoce, aparecerá un cuadro de diálogo preguntando al usuario si desea aceptar o no el certificado.

Encarcelamiento.

Hay un viejo dicho ruso que dice: “Confiar, pero verificar”. Aunque una pieza de software esté firmada, es conveniente verificar que su comportamiento sea correcto. En la figura siguiente se muestra una técnica para hacer esto, conocida como ***encarcelamiento***.

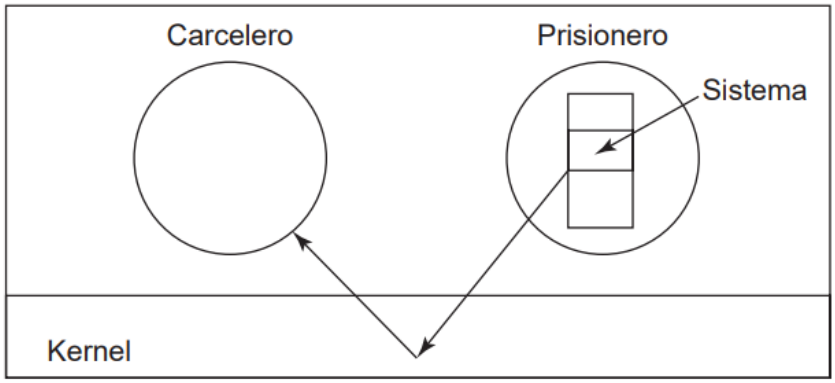


Ilustración 19 - La operación de una cárcel.

El programa recién adquirido se ejecuta como un proceso que en la figura se denomina “prisionero”. El “carcelero” es un proceso confiable (sistema) que monitorea el comportamiento del prisionero. Cuando un proceso encarcelado realiza una llamada al sistema, en vez de que se ejecute esta llamada el control se transfiere al carcelero (por medio de una trampa en el kernel), y se le envía el número de la llamada al sistema junto con sus parámetros.

Después el carcelero decide si se debe permitir o no la llamada al sistema. Por ejemplo, si el proceso encarcelado trata de abrir una conexión de red con un host remoto desconocido para el carcelero, éste puede rechazar la llamada y eliminar al prisionero. Si la llamada al sistema es aceptable, el carcelero informa al kernel y éste lleva a cabo la llamada. De esta forma se puede atrapar el comportamiento erróneo antes de que ocasione problemas.

Existen varias implementaciones del encarcelamiento. Una alternativa en UNIX utiliza las herramientas de depuración normales del sistema operativo, donde el carcelero es el depurador y el prisionero el depurado.

Bajo estas circunstancias, el depurador puede pedir al kernel que encapsule al depurado y le pase todas sus llamadas al sistema para inspeccionarlas.

Detección de intrusos basada en modelos.

Otro método para defender una máquina es instalar un IDS o Intrusion Detection System (Sistema de detección de intrusos).

Hay dos tipos básicos de IDS: uno se concentra en inspeccionar los paquetes de red entrantes y el otro se enfoca en buscar anomalías en la CPU.

En la sección sobre firewalls mencionamos brevemente el IDS de red; ahora hablaremos un poco sobre un IDS basado en host.

Veremos un bosquejo breve de un tipo de IDS, a ese IDS se le conoce como detección de intrusos basada en modelos estáticos, y se puede implementar mediante el uso de la técnica de ***encarcelamiento*** que describimos en la sección anterior, entre otras formas.

En la figura la parte *a* de la siguiente figura podemos ver un pequeño programa que abre un archivo llamado datos y lo lee, un carácter a la vez, hasta que llega a un byte cero y entonces imprime el número de bytes distintos de cero que se encuentran al inicio del archivo, y después termina.

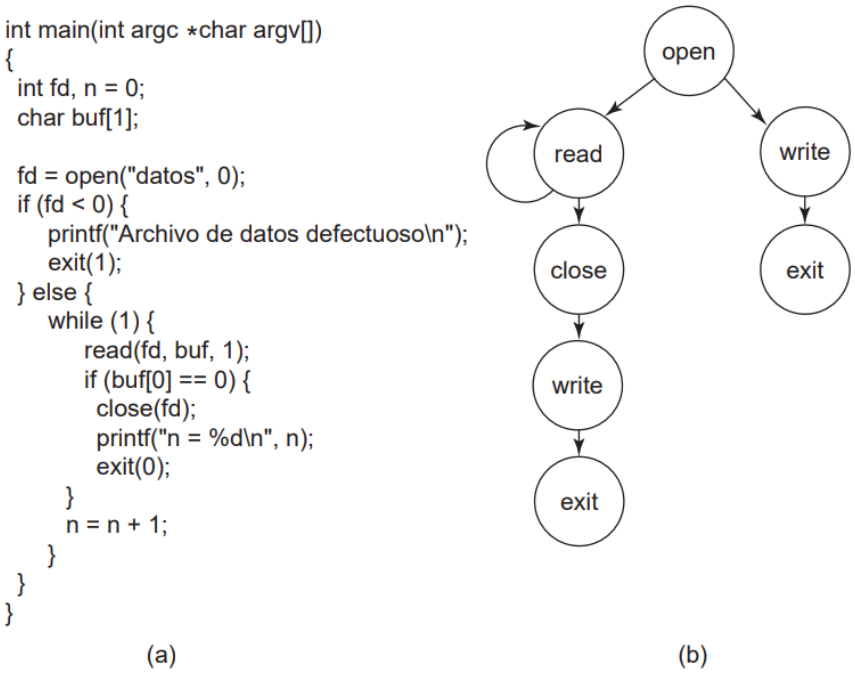


Ilustración 20 - (a) Un programa. (b) Gráfico de llamadas al sistema para (a).

En la parte *b* podemos ver un gráfico de las llamadas al sistema realizadas por este programa (en donde imprimir llama a write).

Para empezar, la primera llamada al sistema que hace el programa bajo cualquier condición es siempre ***open***. La siguiente es ***read*** o ***write***, dependiendo de la bifurcación de *if* que se seleccione.

Si la segunda llamada es ***write***, significa que no se pudo abrir el archivo y la siguiente llamada debe ser ***exit***.

Si la segunda llamada es ***read***, puede haber un número arbitrariamente grande de llamadas adicionales a ***read*** y, en un momento dado, llamadas a ***close***, ***write*** y ***exit***.

En ausencia de un intruso, no hay más secuencias posibles. Si el programa es encarcelado, el carcelero verá todas las llamadas al sistema y podrá verificar con facilidad que la secuencia sea válida.

Ahora supongamos que alguien encuentra un error en este programa y activa un desbordamiento de búfer para insertar y ejecutar código hostil. Cuando se ejecute el código hostil, es muy probable que ejecute una secuencia distinta de llamadas al sistema. Por ejemplo, podría tratar de abrir algún archivo que desee copiar, o podría abrir una conexión de red para telefonear a casa. En la primera llamada al sistema que no encaje en el patrón, el carcelero sabrá en definitiva que ha ocurrido un ataque y puede tomar acción, como eliminar el proceso y alertar al administrador del sistema. De esta forma, los sistemas de detección de intrusos pueden detectar los ataques mientras están ocurriendo.

El análisis estático de las llamadas al sistema es sólo una de las formas en que puede funcionar un IDS. Cuando se utiliza este tipo de detección de intrusión basada en un modelo estático, el carcelero tiene que conocer el modelo (es decir, el gráfico de llamadas al sistema). La forma más simple para aprender esto es hacer que el compilador lo genere, que el autor del programa lo firme y adjunte su certificado. De esta forma, cualquier intento por modificar el programa ejecutable por adelantado se detectará al momento de su ejecución, debido a que el comportamiento actual no coincidirá con el comportamiento firmado esperado.

Por desgracia, es posible que un atacante inteligente lance lo que se conoce como ataque de mímica, en el cual el código insertado realiza las mismas llamadas al sistema que se supone debe hacer el programa, por lo que se necesitan modelos más sofisticados que los que sólo rastrean las llamadas al sistema.

Aun así, como parte de la defensa en profundidad, un IDS puede hacer muy bien su papel. Un IDS basado en modelos no es el único tipo existente. Hay muchos IDS que utilizan un concepto llamado tarro de miel, una trampa para atraer y atrapar crackers y malware.

Por lo general es una máquina aislada con pocas defensas y un contenido en apariencia interesante y valioso, listo para ser recolectado. Las personas que preparan el tarro de miel monitorean cuidadosamente cualquier ataque para tratar de aprender más sobre la naturaleza del mismo.

Algunos IDS ponen sus tarros de miel en máquinas virtuales, para evitar daños en el sistema actual subyacente. Es obvio que el malware tratará de determinar si se está ejecutando en una máquina virtual, como se mencionó antes.

Referencias y bibliografía de la sección.

- Tanenbaum 9-8.